

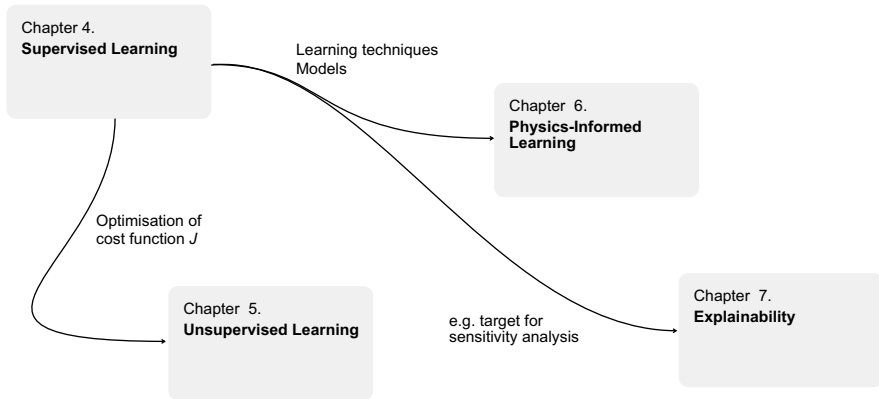


Fig. 4.1 Overview of the relationship of Chap. 4 with the following chapters

means: If event X occurs, then Y also occurs and if Y occurs, then Z follows. **Classical conditioning** is a form of associative learning where a neutral stimulus, e.g. a tone, is linked to an expected reward. Whenever your dog or cat does something you want, you press a clicker and then give a reward in the form of food. This reward is called reinforcement. The animal eventually associates the sound of the clicker with the reward. Hence, this form of learning is also called associative learning.

- **Non-associative Learning.** In contrast to associative learning, non-associative learning deals with exactly one event X —a phenomenon, a noise, or an impression. If this event X occurs often enough, we derive an interpretation from it. The constant noise of water that we gradually get used to, or the decrease of a smell perception that we are exposed to for a long time, are examples of the so-called non-associative **habituation**.
- **Playing.** One of the most impressive and effective learning methods is to play. It is an almost unlimited space for trials and errors, allowing for free development of concept understanding. The ability to play is a fundamental characteristic of intelligent beings.
- **Cultural Learning.** Culture consists of many aspects, with rites, societal rules, laws, and even moral concepts being passed on through culture. This does not always lead to advantageous developments, as the often misguided interpretation of the words “normality” and “naturalness” shows.
- **Imitation Learning.** Role models are people who make a positive impression on us or people we want to emulate. Such emulation is an example of imitation learning. In the animal kingdom, it is observed that older animals pass on certain tricks to young animals to get food. They perform an action and the learners follow it. In the technical field, this is particularly relevant for training robots for production processes, which learn their paths and movement patterns from the user using teach-in techniques.

4.1.2 Various Forms of Machine Learning

Machine Learning can be divided into three forms: supervised, unsupervised, and reinforcement learning.

Supervised Learning deals with the establishment of models f that map input data x to the results d associated with x , $d = f^{(\infty)}(x)$. We assume the availability of d . This gives this type of learning its name supervised: we control towards which result the learning process is trained.

Unsupervised Learning only considers a set of input data. Categorizations or model relationships in the data are unknown. The learning process is now supposed to discover structures in the data. It analyzes whether the data are connected as clusters or to what extent data areas are related to each other.

Reinforcement Learning refers to autonomous agents. These agents have a quantitative goal, their satisfaction. By specifically increasing or decreasing satisfaction, the agent learns whether a decision was good or bad. This type of learning can be trained on online data streams and can be continued continuously. Therefore, it has great relevance for control technology. In fact, we will not return to reinforcement learning in the course of this book, as it does not fit into the framework of this introduction.

4.2 Strategy of Supervised Learning

4.2.1 Algorithmic Consideration

The goal of supervised learning is to map input data $\mathbf{x}$ to output data $\mathbf{y}$, so that they represent a known target size d . Often, d is also called a label—in reference to classification problems. We learn by iteratively changing the model f with $\mathbf{y} = f^{(i)}(\mathbf{x})$, $f^{(i)} \rightarrow f^{(\infty)}$, so that after the change $f^{(\infty)}(\mathbf{x}) \approx d$ results. Here, $f^{(\infty)}$ stands for the converged model, which is realized in practice for very large i or low cost values. To do this, we must be able to evaluate the change of the model f in each iteration.

This evaluation is done via a quality function J , often also called quality functional, cost function or simply cost. If J is large, the model is bad, if it is small, the model is good. If you know the quality function for f , you can decide by comparing this quality whether the model has improved, remained the same or even worsened when changed. The strategy can be divided into the following steps:

- **Setting up an initial model f .** This model is based on initial conditions for the learning parameters and is accordingly bad. It will generate an arbitrary mapping that has no relation to the desired reference results d at first.
- **Calculation of $J(f)$.** Here, the quality of the current model is determined. It then serves as a reference value for the next steps.

- **Changing the model.** We now change f by $f \rightarrow \tilde{f}$. How f changes is dictated by J , because we want J to decrease. This step is critical for learning. The parameters used in the model f are specifically adjusted for the new model $\tilde{f}$.
- **Iteration of the previous step.** We iterate the last step until the costs J have dropped to an acceptable value. This is equivalent to a correspondingly good mapping quality by f .

Supervised learning of a model f describes the iterative change from $f^{(i)}$ to $f^{(i+1)}$, so that

$$f^{(i)} \mapsto f^{(i+1)} \Rightarrow J(f^{(i+1)}) \stackrel{!}{<} J(f^{(i)}) \Leftrightarrow \min_i J(f^{(i)}). \quad (4.2)$$

This simple, universal scheme can be used as the basis for many algorithms. They differ in the type of change step or the choice of cost function J —but almost always follow the procedure outlined in (4.2). Learning algorithms are therefore always optimization problems. This is also evident in (4.2): We are looking for the minimum of J given models $f^{(i)}$. We want to introduce some selected methods in the course of this chapter and briefly summarize them here:

- **Evolutionary algorithms.** In evolutionary algorithms, J corresponds to the reciprocal of a fitness. The variation of the model f occurs through random throws. Whenever J decreases from one step to the next, i.e. the fitness increases, the change is retained. If J increases, the result worsens and the new parameter choice is discarded. Afterwards, one continues to iterate, based on the last good variant. Closely related to this are the genetic algorithms.
- **Least-Mean-Square algorithm.** In the LMS adaptive filter, filter coefficients dynamically adjust so that a target value is achieved by the filter. J is chosen here as the mean square deviation between the target value and the current model result.
- **Neural networks.** Neural networks are nonlinear mappings for which various cost functions J are possible. They train according to the same concept, but mathematically already know the best gradient of J with which they need to change the internal parameters. This method is called backpropagation, and will occupy us in the next sections.
- **Decision trees.** Decision trees search for optimal features in the data to represent a dataset as well as possible. The tree identifies the variables that have the strongest influence on a decision. The tree splits the decision into branches based on these variables and their values. Here too, the quality of the mapping is assessed with the help of a cost function J . It determines the deviation from the target value and specifically adjusts thresholds for the splitting of branches.

4.2.2 *Classification and Regression*

Goal of supervised learning is the creation of a model. This model can represent our input data x at different levels of granularity.

- **Classification.** If the input data should be assigned to a class, then we only distinguish between a few results in the result space. Classifications have discrete output data.

Example: Traffic Light and Motorcycle

An image recognition should recognize motorcycles and traffic lights in videos. The result set is $\Omega = (\text{"Motorcycle"}, \text{"Traffic Light"})$ and $d \in \Omega$. ◀

Example: Alarm Classification

A measurement on a capacitor is examined for voltage peaks. The waveform reflects these peaks. Using learning methods, it is decided whether the signal course is dangerous or not. We use the voltage course as input data and for d we need assignments $d \in \{0, 1\}$ where 0 stands for harmless and 1 for dangerous. This is a canonical classification situation. ◀

- **Regression.** A regression model replicates a complete functional course. The output values are continuous over a defined set of values. Regression models require more data and a minimum resolution in the labels or target values on which the method should be trained.

Example: Probability of Danger

We continue our last example and reformulate it: The evaluation of the signal should not only distinguish between the two categories 1 “Danger” and 0 “No Danger”, but rather output a risk value that, for example, lies between 0 and 1. With a result of $y = 0.8$ a 80 % risk is present. This is a classic example of a regression. ◀

4.2.3 *Training Data and Test Data*

We now define how we want to make the system change according to the existing data and how we calculate the cost function J . At this point, the knowledge of the desired output value d is important. By comparing d and y we can determine the deviation of the current prediction.

To train a supervised learning model, we need sufficient and suitable training data. Sufficient, because we often have to iterate and change in (4.2). Suitable data, as these must always consist of tuples $(\mathbf{x}, \mathbf{y} \rightarrow \mathbf{d})$, where $\mathbf{x}$ represents the input data and $\mathbf{d}$ the desired, ideal mapping result. Subsequently, we will use $\mathbf{y}$ and $\mathbf{d}$ synonymously.

To establish a learning procedure, we need two dedicated data sets, the **training data set**, $\mathbf{x}_{i,\text{Train}} \in X_{\text{Train}}$, $\mathbf{y}_{i,\text{Train}} \in Y_{\text{Train}}$ and the **test data set**, $\mathbf{x}_{j,\text{Test}} \in X_{\text{Test}}$, $\mathbf{y}_{j,\text{Test}} \in Y_{\text{Test}}$. The training data set consists of $|X_{\text{Train}}| = |Y_{\text{Train}}| = N$ data series, the test data set contains $|X_{\text{Test}}| = |Y_{\text{Test}}| = M$ elements. In the code, we always refer to these sets as **Xtrain, Ytrain** and **Xtest, Ytest**.

Since we need more data for training a model, the training data set should contain significantly more data than the test data set, $N \gg M$. Often in practice, a division of 90 % training data and 10 % test data is used. But why are these sets so important? On the one hand, we want to know if the trained learning procedure also works on data that it has never evaluated during training. Only in this way can we ensure in principle that these data will be correctly predicted by the procedure for new data.

The training and test set should also statistically contain all possible cases, scenarios, and classes that you want to learn. If, for example, your training set does not cover all classes of the test set, you would receive distorted test results, since an important part could not be trained at all.

Golden rule for training learning methods: Elements of the test data set must never enter the training!

If you violate the above rule, your test results will be almost perfect without ever being able to transfer the trained model, i.e., apply it meaningfully to new data. This is one of the most common mistakes encountered in practice. We will later learn about the concept of cross-validation, which cleverly mixes the two sets and thus excludes this error.

In practice, it is a great difficulty to set up these data sets for the application of learning methods at all. Not all process databases are designed with the aim of supporting learning methods. Many database structures have grown over the years, and compiling these two data sets takes a lot of time. We have to ask ourselves the following questions: Do we actually have the required labels $\mathbf{d}$ on the data side? Are there enough data in total to train a learning method? These questions must be clarified in a planning manner before the development of a machine learning program.

Another critical point is the process change. Let's assume our learning method is supposed to generate a model f for a process and we actually had enough data. If the process itself has changed while the data was being recorded, the model is useless. We would first have to divide the data into “before the change” and “after the change” split. As you can see, this reduces the amount of data again. All these difficulties are often encountered in the industry.

4.3 Evolutionary Learning

4.3.1 Idea of Evolutionary Change

In evolutionary algorithms, we first talk about populations, in reference to biology. A population describes the parameters $\pi = (\pi_i)$ of our model f , which maps x to y ,

$$y = f(x; \pi). \quad (4.3)$$

Our goal is that for a set of input data (x_j, d_j) the parameters π in f are determined in such a way that $f(x_j; \pi = \pi^*) = d_j \forall j$. Here, j numbers our training set and π^* describes the ideal parameter set. We only discuss the one-dimensional case here, the method can easily be extended to any dimensions.

The fitness F of the Population Π with $\pi_i \in \Pi$ is given by

$$F = \frac{1}{J} = \frac{1}{\sum_j (d_j - y_j)^2}. \quad (4.4)$$

The more d and y match, the higher the fitness F and the lower the cost function J .

What we still lack is the change of π . And here we choose a not very efficient, but extremely instructive variant: we randomize π until the mapping fits.

4.3.2 Implementation of a Simple Evolutionary Algorithm

The following code example shows a simple evolutionary algorithm that tries to find the best result using only random numbers. It is instructive because many brute-force approaches find a solution in a similar way. However, this approach is also inefficient, as one has to wait a long time for a good solution to emerge.

We divide the implementation of an example into four sections. In section (1), we generate artificial training data. They simply represent a normal parabola that we can easily check. Next, we define the function f in section (2) and insert an array p with parameters $p[0]$, $p[1]$ and $p[2]$. In our example, the function describes a 2nd degree polynomial of the form

$$f(x, \pi) = \pi_0 + \pi_1 x + \pi_2 x^2. \quad (4.5)$$

Listing 4.1 A simple evolutionary algorithm

```

1  import numpy as np
2
3  # (1) Input data
4  x = np.arange(0,5,0.1)
5  d = np.arange(0,5,0.1)**2
6
7  # (2) Function f
8  def f(x,p):
9      return p[0]+p[1]*x+p[2]*x**2
10
11 # (3) Definition of fitness
12 def Fitness(d):
13     j = 0
14     for i in range(0,len(d)):
15         j+= (d[i]-f(x[i],p))**2
16     return 1/j
17
18 # (4) Loop for optimizing J and F
19 maxF = 0
20 F = 0
21 while F < 3:
22     p = np.random.random(3)
23     F = Fitness(d)
24     if F > maxF:
25         maxF = F
26     print(maxF)

```

In section (3), the fitness is calculated. We use the square deviation from the target value d . Finally, section (4) contains the iteration steps. π is rolled again each time. With each roll, a new parameterization is given for which a fitness is calculated. As soon as a fitness value is reached that is greater than $F = 3$, the iteration stops. The specific numerical value was found here through tests.

The result of this learning process can be checked well, because for the chosen example, $p[0]$ and $p[1]$ should be as close to 0 as possible and $p[2]$ should be 1. In practice, they find values around 0.04 for the first two parameters and 0.98 for the last one at the chosen limit. The result is then close enough to the data.

4.3.3 Caching the Best Change

In the next code example, we change the view from the fitness F to a view of the cost function J . In addition, we remember the change deviation that brought us the last success in each step. If it was successful, then another step in this direction cannot be wrong.

Listing 4.2 Variation of the evolutionary algorithm

```

11 # (3) Definition of fitness
12 def Fitness(d):
13     j = 0
14     for i in range(0, len(d)):
15         j += (d[i]-f(x[i],p))**2
16     return 1/j, j
17
18 # (4) Loop to optimize J and F
19 J = 1000
20 minJ = 1000
21 p_old = [0,0,0]
22 deviation = [0,0,0]
23 while J > 0.1:
24     p = np.random.random(3) + np.array(deviation)
25     F, J = Fitness(d)
26     if J < minJ:
27         minJ = J
28         deviation = 0.0001*J * (p-p_old)
29     p_old = p

```

But how can we analyze the effect of our intervention in the code? We would need to find an additional quality criterion, one with which we can compare the two variants. To do this, it makes sense to put the number of successful changes in relation to the total number of attempts. In Listing 4.3, we have changed the loop again. We have introduced two counters: `counter` which increments each time a change step was good, and `allCounter` which records every iteration step.

Listing 4.3 Variation of the evolutionary algorithm

```

11 allCounterStorage = []
12 counterStorage = []
13
14 for i in range(0,10):
15     J = 1000
16     minJ = 1000
17     p_old = [0,0,0]
18     deviation = [0,0,0]
19     q = 0
20     a = 1
21     b = 0
22     allCounter = 0
23     counter = 0
24
25     while J > 0.1:
26         p = a * np.random.random(3) + b*np.array(deviation)
27         F, J = Fitness(d)
28         allCounter += 1
29         if J < minJ:
30             counter += 1
31             minJ = J
32             a = 0
33             b = 1
34             deviation = 0.00001*J * (p-p_old)
35             q = 0
36         p_old = p
37         q += 1
38         if q > 10:
39             a = 1
40             b = 0
41             q = 0
42     allCounterStorage.append(allCounter)
43     counterStorage.append(counter)

```

Then the `while` loop was embedded in an outer `for` loop, which realizes exactly ten runs. The reason for this is to generate a little more statistics than a single run for our analysis. The variables `allCounterStorage` and `counterStorage` store the respective counter states for each of the ten runs.

Furthermore, a new form of calculation has been implemented. `p` is first generated randomly. The prefactors are $a = 1$ and $b = 0$, so that really only chance is used. If an improvement of J occurs, we set $a = 0$ and $b = 1$. The deviation is added ten times in a row. After that, $a = 1$ and $b = 0$ are set and a new random value is used.

We can easily compare all variants. It is left to the reader to modify the listing in such a way that only the initial variant from Listing 4.1 is used, which would correspond to a permanent $a = 1$ and $b = 0$. The prefactors in the loop can also be adjusted.

As soon as the ratio of good changes (`counter`) to the total number of iterations (`allCounter`) becomes larger, the algorithm has been formulated more efficiently.

The variation in which we remember the best path of change and follow it for some time is superior to pure chance. This is not surprising. Above all, it shows us that the Least Mean Squares strategy is effective in focusing on the rate of change of J .

4.4 LMS Adaptive Filter

4.4.1 Theory of the LMS Algorithm

The next learning algorithm we want to discuss is the Least-Mean-Squares adaptive filter. Fundamental works on this topic come from Haykin et al. [6] and Widrow et al. [15], where in addition to filter-theoretical derivations, stability investigations and convergence criteria are described. The adaptive filter has also been successfully used for learning language, as shown by Koike in 1997 [9].

As the name already suggests, this learning algorithm comes from the field of filter theory. Let's assume we have a vector $\mathbf{a} \in \mathbb{R}^N$ of dimension N . Please imagine, without loss of generality, that the entries of this vector are completely random at the beginning. Our input data is another vector $\mathbf{x} \in \mathbb{R}^N$ of the same dimension. Now, $\mathbf{a}$ acts like a filter on this input to produce a scalar result $y \in \mathbb{R}$ —our output data—,

$$\mathbf{a}^T \mathbf{x} = y. \quad (4.6)$$

The effect of $\mathbf{a}$ on $\mathbf{x}$ is formulated here in (4.6) as the scalar product of the two vectors. We use the symbol T for the transposition of the vector $\mathbf{a}$. We want $\mathbf{a}$ to “learn” to map $\mathbf{x}$ to a defined, desired target number d . We use the symbol d in reference to the English literature, which also likes to refer to this value as *desired value*. So we ask the question: “How does $\mathbf{a}$ have to change for this?” This change is implemented in the form of several iteration steps, which we capture using the notation $\mathbf{a}^{(0)}, \mathbf{a}^{(1)}, \dots, \mathbf{a}^{(i)}$, where i denotes the i -th iteration. If we use a transposition here, we show this with $\mathbf{a}^{(i),T}$ on. The first step is to find out how far d deviates from the current result y , i.e., to calculate the error $\varepsilon^{(i)}$,

$$\varepsilon^{(i)} = d - \mathbf{a}^{(i),T} \mathbf{x}^{(i)}. \quad (4.7)$$

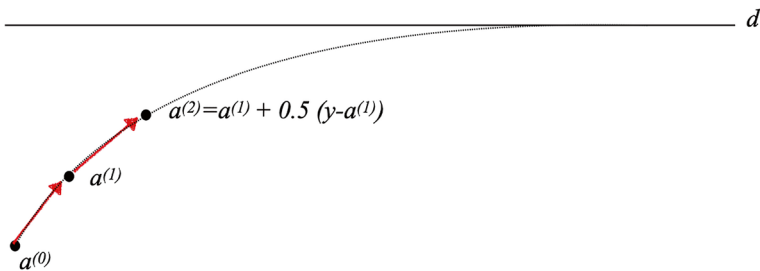


Fig. 4.2 Various iterations of $\mathbf{a}^{(i)}$ and approximation to the preset value d . A learning rate of 0.5 was chosen as examples

The error also refers to the current iteration and is marked accordingly, as it changes with every change of $\mathbf{a}$. We can read a lot from this error. If the product $\mathbf{a}^{(0),T} \mathbf{x}^{(0)}$ is, for example, larger than d , then $\mathbf{a}^{(0)}$ is too large and needs to be reduced. The same applies in reverse. The error therefore contains the instruction on how strongly and in which direction we should change $\mathbf{a}^{(0)}$.

We need a way to find out this instruction. To do this, we make the following demand: With each iteration, the square of the error should become smaller. Formally, one writes (Fig. 4.2),

$$J^{(i)} = (d - \mathbf{a}^{(i),T} \mathbf{x}^{(i)})^2, \quad (4.8)$$

and demands

$$\lim_{i \rightarrow \infty} J^{(i)} \stackrel{!}{=} 0. \quad (4.9)$$

If J is to move quickly towards such a minimum, we only need to determine the negative gradient of J with respect to $\mathbf{a}^{(0),T}$. The gradient, which we write over ∇J with the Nabla operator, always points in the direction of the steepest increase, consequently the negative gradient points in the direction of the steepest descent. So we derive (4.8) according to the individual entries of $\mathbf{a}$,

$$-\nabla_{\mathbf{a}^{(i),T}} J = -\nabla_{\mathbf{a}^{(i),T}} (d - \mathbf{a}^{(i),T} \mathbf{x}^{(i)})^2 = 2 \varepsilon^{(i)} \mathbf{x}^{(i)}. \quad (4.10)$$

Please note that we derive over the operation $\nabla_{\mathbf{a}^{(i),T}}$ according to the values of $\mathbf{a}^i$ and we use the chain rule in this derivation step. We want to change the values of $\mathbf{a}$ according to (4.10) to minimize J , and we implement this change in the following equation:

$$\mathbf{a}^{(i+1)} = \mathbf{a}^{(i)} + \lambda \varepsilon^{(i)} \mathbf{x}^{(i)}. \quad (4.11)$$

It contains a rule for the iteration $i + 1$. The dimensionless factor λ also captures the factor 2 from the calculation in (4.10) and is called **learning rate**. If the learning rate is small, then $\mathbf{a}^{(i)}$ only slowly approaches d with each iteration, if it is larger, the approximation is carried out faster. Fig. 4.2 shows the convergence of the method.

Due to its derivation from the square of the error in (4.8), this approach is also called Least-Mean-Squares (LMS) adaptive filter. We will always refer to this as the LMS algorithm from now on.

4.4.2 Implementation of the LMS Algorithm

We now consider how we can use (4.11) in the code. For this, we have created a code example in Listing 4.4. If you run this, y quickly reaches values close to d . We can consider this example as the simplest form of a learning program.

Listing 4.4 Implementation of the adaptive filter in Python

```

1  import numpy as np
2
3  a = np.random.random(3)
4  x = [1,2,3]
5  d = 1.0
6  eps = d - np.dot(a,x) # Fehler
7
8  Lambda = 0.001 # Lernrate
9  while eps**2 > 0.00000001:
10     a = np.array(a) + Lambda*eps*np.array(x)
11     eps = d - np.dot(a, x)
12
13 y = np.dot(a,x)
14 print('y=', y)

```

4.4.3 NLMS Algorithm by Normalizing the Input Variables

In fact, the implementation in Listing 4.4 is not stable. As soon as you use higher values for λ , abbreviated in the code by l , the calculation of y will diverge and not yield a result. It is therefore difficult to define a suitable learning rate for the respective problem so that the algorithm always converges. The reason for this is the influence of $\mathbf{x}$, whose magnitude affects the calculation quadratically over line 10—via the product $\varepsilon \mathbf{x}$, where ε also contains $\mathbf{x}$ once again. From a stability point of view, it makes sense to normalize $\mathbf{x}$ by the substitution

$$\mathbf{x} \rightarrow \tilde{\mathbf{x}} = \mathbf{x}/(\mathbf{x}^T \mathbf{x}) \quad (4.12)$$

. From (4.11) the learning rule

$$\mathbf{a}^{(i+1)} = \mathbf{a}^{(i)} + \lambda \frac{\varepsilon^{(i)} \mathbf{x}^{(i)}}{(\mathbf{x}^{(i),T} \mathbf{x}^{(i)})}. \quad (4.13)$$

is then derived. In the code example, this only leads to a change in line 10, as only the scalar product of $\mathbf{x}$ is divided by itself. This line is listed in Listing 4.5.

Listing 4.5 Alternative line 10 for Listing 4.4

```

10 | a = np.array(a) + l*eps*np.array(x)/np.dot(x,x)

```

While the code in Listing 4.4 represents an implementation of the LMS algorithm, 4.5 is referred to as the NLMS algorithm, where N stands for normalized.

4.4.4 Training

The while loop implemented in lines 9–11 continuously applies Eq. (4.13). Each new loop is another iteration. In the code example, we do not explicitly store the

individual iterations, but overwrite the value for $\mathbf{a}$. The variable $\mathbf{x}$ remains constant for simplicity and does not change with the iteration.

The process that the `while` loop represents is a concrete example of training a learning procedure. The LMS algorithm and the NLMS algorithm are particularly well suited to understanding the approach of supervised learning programs. The algorithm learns supervised because it has a specification with d on which it should train.

4.4.5 Applications and Properties of the Adaptive Filter

The adaptive filter has many technical applications. It allows to dynamically respond to changes and quickly adapt to these changes. Often the target specifications are formulated more complex than in our derivation. However, these are always comprehensible extensions that build on the basic concept above.

Example: Speed Controller

If you give a target value to a vehicle's cruise control, the speed controller uses the strategy in (4.11) to iteratively adapt to the specification. If the target value was, for example, $d = 50$ km/h and the current speed $v = 20$ km/h, then the acceleration would be large at the beginning and decrease as it approaches the target value through the filter. ◀

However, there are also weaknesses of the algorithm that we would like to briefly explain here.

- Variability of x . If x changes quickly from iteration to iteration, the basic idea is void and the circumstances are so unfavorable that (4.11) no longer works. If the input values fluctuate more than an adjustment via λ is possible, then the strategy must fail. The filter can no longer keep up with the change in inputs.
- Variability of d . This algorithm also has problems if the target size should change.

4.4.6 Hyperparameters of the LMS Adaptive Filter

Obviously, the value of λ plays a crucial role in (4.11). It is the only obvious parameter of the algorithm. A less obvious parameter is the temporal distance of the iterations, i.e., the time that passes between i and $i + 1$. It was not addressed in the derivation of (4.11), but it has an impact on the algorithm.

Whenever we define quantities that have an influence on the effect and quality of an algorithm, we speak of **hyperparameters**. The choice of these parameters is crucial for the success of our procedure.

Factors that we can adjust or choose in a learning procedure and that have a significant influence on the quality of the procedure in terms of learning speed, model quality, and stability are called **hyperparameters**.

4.5 Neural Networks

4.5.1 The Neuron

Neural networks are nonlinear mappings of an input x to an output y . They allow us to learn relationships from data. To understand such a network, we first formulate what we understand by a neuron:

Let $\mathbf{x}$ be a vector of input values, then the function

$$\mathcal{N}_k(\mathbf{x}) = f \left[\sum_{i=0}^{N-1} w_{ik} x_i + b \right], \quad (4.14)$$

is called the k -th **neuron** with the weights w_i , the threshold b and the activation function f .

A neuron thus calculates a state value from N inputs x_i . This is the result $\mathcal{N}(\mathbf{x})$ of (4.14) and is passed on to subsequent neurons. To calculate the state, the inputs are multiplied by weight factors w_i . They reflect the influence of each individual i -th input. We also call the sum formation the input through the network itself and abbreviate this with

$$\text{net}_k = \sum_{i=0}^{N-1} w_{ik} x_i \quad (4.15)$$

The structure of a neuron is shown in Fig. 4.3.

The bias b is a parameter that can be used to change the effect of the activation function. It shifts it by a constant value. If b is high, it is more difficult to activate the neuron. If it is low, it is easier to activate. Ultimately, the so-called activation function f is applied to the weighted inputs and the threshold. There are many different functions available, which we will discuss in more detail later.

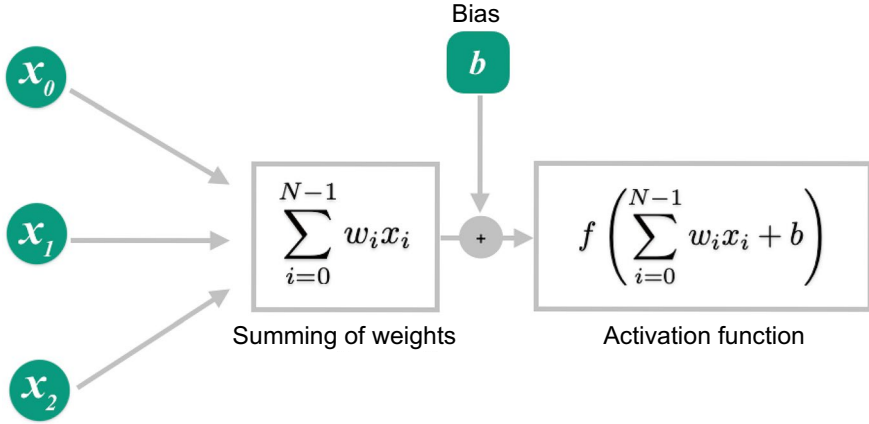


Fig. 4.3 Abstract illustration of a neuron, the smallest component of neural networks

4.5.2 Networking of Neurons

Neurons can be connected together. The activation state of a neuron is used as an input for the next connected neuron. Fig. 4.4 shows a simple network of five neurons. The network shown consists of two **layers**, the input layer with $N = 3$ neurons and the output layer with $M = 2$ neurons. The input layer represents an exception to the above definition, here the excitation of the neurons is not calculated, rather the respective values are the actual input of the network.

In general, the neurons can be linked arbitrarily, but the example shown only connects the neurons of different layers. No connection is used within a layer. Such a network architecture is called a **Multi-Layer Perceptron**. It represents a simple, frequently used form of neural networks.

For this network, we can calculate the output neurons y_0 and y_1 . The input neurons are linked to the output neurons via the weights w_{ij} . This layer connection can thus be understood as a matrix, where the row number stands for the target neuron (here 0 or 1) and the column indicates from which original neuron the input comes. In Fig. 4.4, these weights are accordingly drawn onto the connection lines.

From the definition of the neuron, the rule with which we can calculate the excitation of neurons of the k -th layer follows directly,

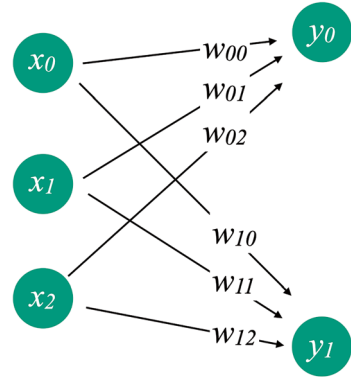
$$y_k = f\left(\sum_{i=0}^{N-1} w_{ki}x_i + b_k\right), \quad (4.16)$$

which can be written vectorially as

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad (4.17)$$

with the weight matrix $\mathbf{W}$,

Fig. 4.4 Simple neural network with three neurons in the input layer and two neurons in the output layer



$$\mathbf{W} = \begin{pmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \end{pmatrix}, \quad (4.18)$$

and the threshold vector (engl. *bias*) $\mathbf{b}$,

$$\mathbf{b} = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}. \quad (4.19)$$

From this simple case, we extend our discussion to more complex networks. For illustration, a network with four layers is sketched in Fig. 4.5. For each layer transition, there is a weight matrix $\mathbf{W}_i$. Layer 0 is the input layer, layer 3 represents the output of the network.

In a **Multi-Layer Perceptron** with k layers, the input layer $\mathbf{x}_0$, the weight matrices $\mathbf{W}_k$ and the threshold vectors $\mathbf{b}_k$, the values of the subsequent layers can be calculated with $k > 0$ over

$$\mathbf{x}_k = f_k(\mathbf{W}_{k-1}\mathbf{x}_{k-1} + \mathbf{b}_k) \quad (4.20)$$

calculate, if f_k is the activation function of the k -th layer. We also refer to this calculation as the forward evaluation of the network.

Along the line in Fig. 4.5, the network is thus evaluated **forward**. One iteratively calculates from $\mathbf{x}_0$ the next layer $\mathbf{x}_1$ and then continues until the last layer has been determined.

Although we now know how to describe a network abstractly and calculate it from the network input to the output, we have not yet explained how the matrices $\mathbf{W}_k$, the threshold vectors $\mathbf{b}_k$ and the activation functions f_k can be found.

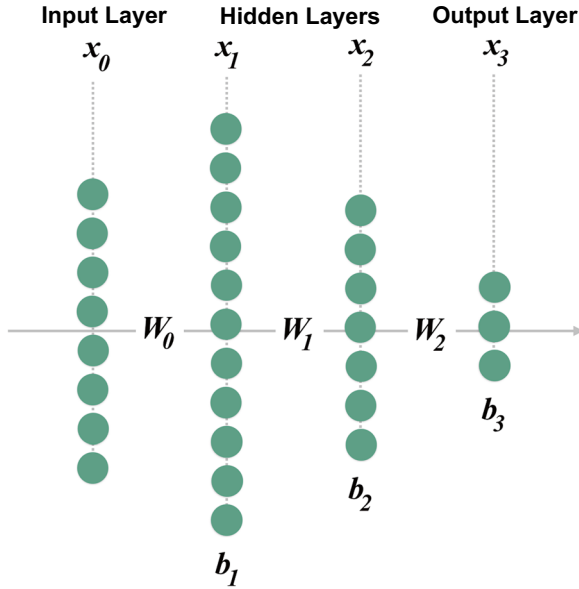


Fig. 4.5 Neural network with multiple layers, weight matrices, and threshold vectors

In the textbooks by Ripley [13] and [1], neural networks are explained with great theoretical depth and for various other forms of realization, so-called topologies. We will limit ourselves here to the exemplary implementation of simple networks in Tensorflow and Keras, as they are most relevant for technical application. For all deeper considerations, the above works are recommended.

4.5.3 Activation Functions

To give you an overview of the possible activation functions, we will now summarize some selected functions. This compilation is for information only, as we can retrieve them later via Python libraries without having to program them ourselves.

- **Sigmoid Function.** The Sigmoid function is one of the most well-known activation functions for neurons. It is given by

$$\text{sig}(x) = \frac{1}{1 + \exp(-x)} \quad (4.21)$$

and has a value range between 0 and 1.

- **Hyperbolic Tangent.** Similar to the sigmoid function in terms of its course, $\tanh$ maps the input $x \in \mathbb{R}$ over the

$$f(x) = \tanh x = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)} = 2 \operatorname{sig}(2x) - 1 \quad (4.22)$$

to the value range -1 to 1 . Thus, such neurons can take on negative and positive values. We have also listed the relationship with the sigmoid function in the equation.

- **Rectified Linear Unit (ReLU).** An activation function that is successfully used in many current neural networks is the ReLU function:

$$\operatorname{relu}(x) = \begin{cases} x & \text{for } x \geq 0 \\ 0 & \text{else.} \end{cases} \quad (4.23)$$

It is linear for positive x and 0 for negative x . This fact makes it particularly easy to calculate its derivative, which is always 1 for positive x and 0 in all other cases. ReLU is excellent for hidden layers.

- **Leaky-ReLU.** An important variant of the ReLU function is the Leaky-ReLU. This avoids a training situation where a neuron is trained to death, i.e., it has no gradient to contribute to training for a longer period. The function varies ReLU as follows,

$$\operatorname{leaky}(x) = \begin{cases} x & \text{for } x \geq 0 \\ \varepsilon x \text{ mit } \varepsilon \ll 1 & \text{else} \end{cases} \quad (4.24)$$

and thus always introduces a positive gradient, even in the range of negative x .

- **Softmax.** Specifically for dealing with the output layer in classification problems, an activation function is needed that allows for probability statements. This function is called Softmax and is defined as follows,

$$\operatorname{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}, \quad (4.25)$$

where this activation function accesses all neurons of the output layer to calculate the sum in the fraction.

In the further course, we will get to know other activation functions. How do we choose the right activation function for a neural network or for the individual layers? There is no universally valid recipe. Many networks are optimized through tests and variations until a good mapping quality and low error have been achieved. Nevertheless, some rules can be set up that can help to some extent in developing the architecture of a network:

- If the output layer must reflect numerical values greater than 1, pure sigmoid or $\tanh$ functions are not suitable. In such cases, the use of ReLU often helps.
- For classification problems, the output layer should be activated with Softmax.

4.5.4 Training of Neural Networks

How in the case of the adaptive LMS algorithm, a neural network learns through the comparison of output and reference. We determine the error of the network by calculating an output value $\mathbf{x}_k$ with its current weights and thresholds and comparing this with the given target value $\mathbf{d}$. Note that in the general case, the network output is multidimensional and thus also the training specification $\mathbf{d}$. The training assumes that a training set with sufficient inputs $\mathbf{x}_0$ and desired results $\mathbf{d}$ is available. Here too, we determine a quality functional J , to quantify the deviation of the network output $\mathbf{x}_k$ from the desired result $\mathbf{d}$. A simple example of such a quality functional is (again) the square distance,

$$J = (\mathbf{x}_k - \mathbf{d})^2. \quad (4.26)$$

There are other quality or cost functions that we could use for J . To explain the training of the network, however, we initially stick with this cost function. To train the network, we iterate over the training set. We systematically adjust the weights and thresholds so that J becomes smaller. If we minimize J , the network maps the input $\mathbf{x}_0$ to $\mathbf{x}_k \approx \mathbf{d}$. The network learns the relationship between $\mathbf{x}_0$ and $\mathbf{d}$. To reduce J as efficiently as possible, we need to know how we can adjust the weights. The weights influence J in (4.26) via y ,

$$J = [f(\mathbf{W}_k \mathbf{x}_k + \mathbf{b}_y) - \mathbf{d}]^2. \quad (4.27)$$

The derivative of the cost function with respect to the weights

$$\frac{\partial J}{\partial w_{ik}} = \frac{\partial J}{\partial f} \frac{\partial f}{\partial \text{net}_k} \frac{\partial \text{net}_k}{\partial w_{ik}}, \quad (4.28)$$

so the gradient of J with w_{ik} , points in the direction of the steepest increase of J with that very weight. As we have already seen in (4.10), we now only need to move in the direction of the steepest descent (i.e., the negative gradient) of J to deliberately minimize J . This is also referred to as the gradient descent method. We adjust each weight w_{ik} therefore via this gradient by changing it by

$$\Delta w_{ik} = -\eta \delta_i x_k \quad (4.29)$$

where δ_i is given by

$$\delta_i = \begin{cases} f'_i(\text{net}_i)[x_i - d_i] & \text{if } i \text{ is the output layer} \\ f'_i(\text{net}_i) \sum_j \delta_j w_{ij} & \text{else} \end{cases} \quad (4.30)$$

Specifically for the inner layers, we need to change the weights sequentially. We do this by starting from the output and moving forward through the network,

changing the weights. This process is called **Backpropagation** and trains the network. For specific activation functions f , the change Δ_{ik} over (4.30) depends on the derivative f' .

4.5.5 Optimization as Hyperparameter

Already in our introduction, we mentioned the relationship between optimization and machine learning. The neural network optimizes the function J in such a way that it is minimized for a given set of training data. The derivation of Δw_{ik} in the previous section is illustrative. It explains to us how to analytically bring about the change in weights when J is defined as a square distance. The most important ingredient in (4.29) was the gradient of J from (4.28) and the gradient descent method. For gradient descent, optimized variants exist that we can use via the Python libraries:

- **Stochastic Gradient Descent** [14] (engl.). While the gradient descent method uses the average of all available training data for its cost calculation, the SGD selects a subset of training data via a random generator. This new training set is smaller and the learning process is accelerated. The advantage of SGD is that this optimizer can also be used on very large data sets.
- **ADAM** [8], **ADAGRAD** [3] and **RMSProb** [12]. These three variations are based on the SGD. **ADAGRAD** varies the learning rate of the individual weights and leads to adaptive gradients. Frequently changed weights are assigned low learning rates and rarely changed weights are assigned high learning rates. This step is helpful to increase the convergence speed of the method. **ADAM** and **RMSProb** ensure that too high gradients have less influence and low gradients contribute more to the learning process.

4.5.6 Simple Classification Network with Scikit-Learn

We begin our implementation examples with a simple, pre-made neural network from the Scikit-Learn library. This collection of learning methods is excellent for getting to know the individual methods, experimenting with them, and setting up initial models.

But first, we load our data from the motor current example. Listing 4.6 shows how you can load and display this from the pickle file.

Listing 4.6 Loading and visualizing the data

```

1 import matplotlib.pyplot as plt
2 import pickle
3
4 data = pickle.load(open('EX03Engine.pickle','rb'))
5 X = data['X']
6
7 mycolor = []
8 for i in range(0,500):
9
10     if data['Label'][i] == 1:
11         mycolor.append('r')
12     elif data['Label'][i] == 2:
13         mycolor.append('k')
14     elif data['Label'][i] == 3:
15         mycolor.append('b')
16     else:
17         mycolor.append('y')
18
19     plt.plot(X[i], color=mycolor[i],alpha=0.1)
20
21 plt.show()

```

After we have loaded the test data, we manually divide it into a training and a test set. In Listing 4.7 the arrays `Xtrain`, `Ytest` as well as `Xtest` and `Ytest` have been created for this purpose. Since we have a label in the dataset, the assignment here is particularly simple.

Listing 4.7 Splitting into training and test sets

```

1 Xtrain = []
2 Ytrain = []
3
4 Xtest = []
5 Ytest = []
6
7 for i in range(0,1000):
8     Xtrain.append(X[i])
9     Ytrain.append(data['Label'][i])
10
11 for i in range(1401,1420):
12     Xtest.append(X[i])
13     Ytest.append(data['Label'][i])

```

With the two data groups, we can now train a neural network. Our first step is to include a Multi-Layer Perceptron from the Scikit-Learn library. This can be

included via `sklearn.neural_network.MLPClassifier` as Listing 4.8 shows. The call requires us to create an object `classifier`, to which we assign a corresponding `MLPClassifier` with special properties. In the present code example, the optimizer “lbfgs” is selected, the learning rate is initialized with 0.01 and in addition to an input and an output layer, a single hidden layer with 25 neurons is specified. The network is manageable and flat.

Listing 4.8 Neural network using Scikit-Learn

```
1 from sklearn.neural_network import MLPClassifier
2
3 classifier = MLPClassifier(
4     solver='lbfgs',
5     learning_rate_init=0.01,
6     hidden_layer_sizes=(25,)
7 )
8
9 classifier.fit(Xtrain, Ytrain)
```

After training the network, the immediate question is how good the model is. This is reflected in the cost function in the model, but we now want to test our model directly on the test data. For this, the function `predict` of the `MLPClassifier` class can be used. A simple comparison on the test set, as it is also helpful for other methods, is shown in Listing 4.9.

Listing 4.9 Test procedure for showing classification results

```
1 result = clf.predict(Xtest)
2
3 print('NN_|_Test')
4 for i in range(0,19):
5     print('{ }_|_{ }'.format(result[i], Ytest[i]))
```

4.5.7 Classification Network in Keras

Individual Network Topology and Optimization

As simple as the implementation with the help of Scikit-Learn is, if you want to create more complex structures or even interact more deeply with the network, then the implementation with the help of the Tensorflow and Keras libraries is recommended. Keras gives us access to simple structures for programming models and Tensorflow contains fast, GPU and neural core optimized training algorithms.

Especially the previously discussed derivatives of activation functions and the optimization algorithms are included in Tensorflow. This leads to efficient and fast training processes. However, these advantages also mean that we have to set up a network in a more complicated way.

Categorical Mapping

One tool for developing a neural network for classification is categorical mapping. With a single neuron in the output, we can easily classify the states 0 and 1. But if there are N many, discrete categories, then we also need N neurons. Each neuron then stands for a class.

The categorical mapping is a rule

$$x \mapsto \sum_i \delta_{ix} \mathbf{e}_i, \quad (4.31)$$

that assigns a unit vector $\mathbf{e}_i$ with the position x to a scalar classification number x .

In practice, the following assignment explains how this regulation works:

$$\begin{aligned} 0 &\mapsto [1, 0, 0, 0, 0], & 1 &\mapsto [0, 1, 0, 0, 0], & 2 &\mapsto [0, 0, 1, 0, 0], \\ 3 &\mapsto [0, 0, 0, 1, 0], & 4 &\mapsto [0, 0, 0, 0, 1] \end{aligned} \quad (4.32)$$

In Keras, there is a helper function that takes this step for us. This function is called `to_categorical()` and ensures that our categories are mapped to vectors with 0 and 1 accordingly.

Implementation of the classifier

Listing 4.10 shows a code example for classification with Keras. We need the model class from Keras. From this, our own Python object inherits all necessary properties and functions, here `class Classifier(Model)`. Note that we also need to initialize the model itself, so there is a corresponding call to the constructor of the superclass in line 9.

Listing 4.10 Classification network in Keras

```

1  import keras
2  import numpy as np
3  import tensorflow as tf
4  from keras.utils import to_categorical
5  from tensorflow.keras import layers, losses
6  from tensorflow.keras.models import Model
7
8  class Classifier(Model):
9
10     def __init__(self, inputLayerLength, hiddenLayers=2):
11         super(Classifier, self).__init__()
12         self.inputLayerLength = inputLayerLength
13         self.hiddenLayers = hiddenLayers
14         self.constructLayers()
15         self.classifier = tf.keras.Sequential(self.myLayers)
16         self.compile(optimizer='adam',
17                     loss=losses.CategoricalCrossentropy())
18         self.optimizer.learning_rate = 0.001
19
20     def constructLayers(self):
21         self.myLayers = []
22         self.myLayers.append(layers.Input(self.
23             inputLayerLength))
24         for i in range(0, self.hiddenLayers):
25             self.myLayers.append(layers.Dense(50, activation='
26                 relu'))
27         self.myLayers.append(layers.Dense(5, activation='
28             softmax'))
29
30     def call(self, x):
31         classified = self.classifier(x)
32         return classified
33
34 classifier = Classifier(len(Xtrain[0]))
35 history = classifier.fit(np.array(Xtrain),
36                         to_categorical(np.array(Ytrain)),
37                         epochs=50, batch_size=50)

```

By calling the function `to_categorical(Ytrain)`, our categorization, as described above, is encoded on five neurons, which corresponds to the topology with 5 neurons in the output layer.

In the further course, this code example shows another feature: the construction of the layers is outsourced to an additional member function `constructLayers` of the class. Here you will find the input layer, the hidden layers, and the output layer. A ReLU function was specified as activation in the hidden layers. Since it is a classifier, a softmax activation function is used in the output layer. The chosen optimizer is ADAM and the cost function selected is the categorical cross-entropy. The initial learning rate is 0.001 in this case.

Epochs and Batch Size

For the call of the training, we must specify two parameters: the number of epochs and the batch size. An epoch stands for training on a dedicated data package, the batch, whose size is determined by the batch size. A small batch size thus means fast training of an epoch. For each epoch, the mix of data in the batch is randomly selected from the training data set `Xtrain`. Each epoch results in a final cost value for J .

Lines 30–33 finally specify this call. The training proceeds similarly to the example with Scikit-Learn. As a result, we get a classification model, which can be tested as in Listing 4.9. However, we need to vary our test because we have a prediction from 5 neurons whose activation tells us which category we have in the result. Listing 4.11 shows the test again, now with a modification in line 5: here the `argmax` of the classification result is now returned.

Listing 4.11 Test of the Keras classification network

```
1 result = classifier.predict(np.array(Xtest))
2
3 print('NN_|_Test')
4 for i in range(0,19):
5     print('{}_|_{}'.format(np.argmax(result[i]), Ytest[i]))
```

Feel free to use this code to test different optimizers (SGD, RMSprop) and activation functions (e.g., ReLU or Leaky-ReLU).

Learning curve

In line 32 of Listing 4.10 we have stored the variable `history`. It contains all information about the learning process, including the course of costs in `history.history['Loss']`. This course is called the learning curve. In Fig. 4.6 the learning curve for the classifier training with the SGD optimizer is shown.

4.5.8 Regression network in Keras

Next, we want to train a regression network. For technical processes, regression is an established tool. Often, defined functions are used to set up parametric models with them. Polynomial regression is a prominent example of this.

To set up a simple initial model, we need a set of data. We want to keep this simple and therefore generate synthetic data ourselves. For this, we use a polynomial of the 2nd degree as a basic framework. The coefficients of the polynomial p fluctuate around a base value defined by us $p = (0.5, 1, 0.5)$.

Listing 4.12 shows the creation of the training and test set. In the function `polynom(t, p)`, the actual polynomial is calculated. The function `generateSyntheticData(t, n)` generates n different polynomials each with the parameters p_i . The function plots represent our input data. They are stored in `Xtrain`

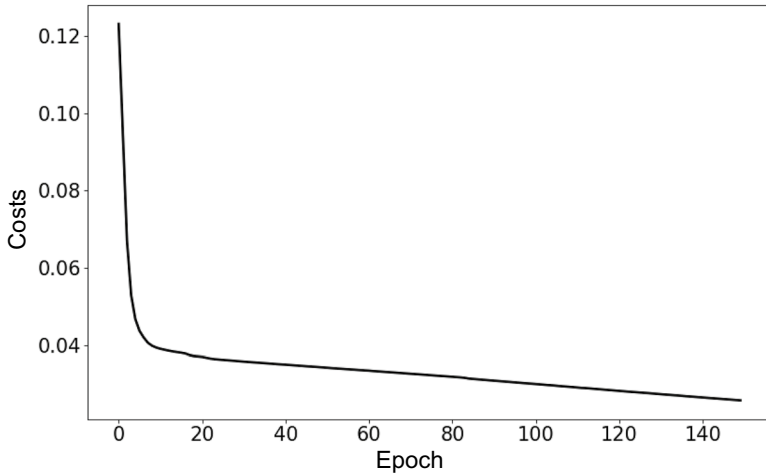


Fig. 4.6 Learning curve of the classification network

and `Xtest`. We save the strength of the quadratic component as our regression targets—i.e., our desired results—for both the training set `Ytrain` and the test set `Ytest`.

Listing 4.12 Synthetic data for a regression network in Keras

```

1  def polynom(t, p):
2      f = 0
3      for i in range(0, len(p)):
4          f += p[i] * t ** i
5      return f
6
7  def generateSyntheticData(t, n):
8      result = []
9      target = []
10     p0 = [0.5, 1, 0.5]
11     for i in range(0, n):
12         p = p0 + 0.5 * np.random.random(3)
13         result.append(polynom(t, p))
14         target.append(p[2])
15     return result, target
16
17  Xtrain, Ytrain = generateSyntheticData(np.arange(0, 5, 0.2), 500)
18  Xtest, Ytest = generateSyntheticData(np.arange(0, 5, 0.2), 12)
19
20  for i in range(0, len(Ytrain)):
21      Xtrain[i] = Xtrain[i] - np.mean(Xtrain[i])
22
23  for i in range(0, len(Ytest)):
24      Xtest[i] = Xtest[i] - np.mean(Xtest[i])

```

Please also note that we subtract the mean values from the function plots for both data sets. The training data generated in this way are now used as the basis for a neural regressor. This regressor is written very similarly to the above classifier. Listing 4.13 shows the code for this network.

Listing 4.13 Regression network in Keras

```

1  class Regressor(Model):
2
3      def __init__(self, inputLayerLength, hiddenLayers=10):
4          super(Regressor, self).__init__()
5          self.inputLayerLength = inputLayerLength
6          self.hiddenLayers = hiddenLayers
7          self.constructLayers()
8          self.regressor = tf.keras.Sequential(self.myLayers)
9          self.compile(optimizer='adam',
10                      loss=losses.MeanSquaredError())
11          self.optimizer.learning_rate = 0.001
12
13      def constructLayers(self):
14          self.myLayers = []
15          self.myLayers.append(layers.Input(self.
16                                           inputLayerLength))
17          for i in range(0, self.hiddenLayers):
18              self.layers.append(layers.Dense(10, activation='
19              relu'))
20          self.myLayers.append(layers.Dense(1, activation='
21          leaky_relu'))
22
23      def call(self, x):
24          regressorResult = self.regressor(x)
25          return regressorResult

```

Please note the changes compared to the classifier. The cost function (English *loss*) has been adjusted and the mean square error has been used. We want to minimize this as much as possible during training. The categorical distance as in a classifier would not be able to do this. The output layer has only one neuron, which should take the value of the quadratic coefficient of our polynomial. As activation functions, we combine ReLU and Leaky-ReLU. The learning rate is set to 0.001. In addition, we use ADAM as the optimizer for minimizing the cost function.

With the command in Listing 4.14 we start the training.

Listing 4.14 Running the regression network

```

1  regressor = Regressor(len(Xtrain[0]))
2  history = regressor.fit(np.array(Xtrain), np.array(Ytrain),
3                          epochs=350, batch_size=100)

```

In a final step, we need to test the quality of the model. For this, we evaluate the regression network on the test inputs *Xtest* and compare the results of the model prediction with the true values *Ytest*, which exactly reflect our target parameters. A corresponding code for the test is given in Listing 4.15.

Listing 4.15 Plotting the results of the regression network

```

1 plt.plot(Ytest, 'k')
2 plt.scatter(range(0, len(results)), results, s=60, color='k')

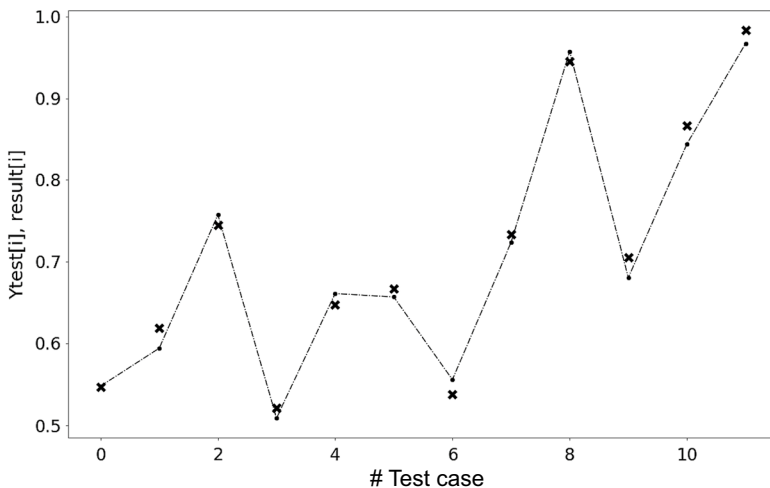
```

The result of this test is shown in Fig. 4.7. Points indicate the actual expected values, with the fine connecting line merely helping the eyes to better follow the distribution of the points (as there are no data in between). The results of the regressor are marked with the symbol $\times$. The reconstruction of the quadratic part of the polynomial is extremely good in this case.

4.5.9 Overfitting and Cross-Validation

It can happen, that a learning process, not necessarily only neural networks, overfits a dataset. This means that the process no longer trains only meaningful differences in the input data, but also learns the noise in the input data. However, this noise does not carry any additional information. Above all, it can no longer help to improve the learning results on the test data. In Fig. 4.8 (a) shows an ideal classification and (b) graphically illustrates the problem of overfitting. Due to the overfitting, the error during training decreases, leading us to the erroneous assumption that our prediction quality is constantly improving.

We can detect overfitting by comparing the training error with the error on the test data. Overfitted learning algorithms show a low error on the training data and a significantly higher one on the test data. For the application, this means that one trains, then looks at the error on both datasets, and as soon as the error on the training data significantly drops below the error on the test data, one is in a state of overfitting.

**Fig. 4.7** Result of the regression network for predicting the quadratic part of a polynomial

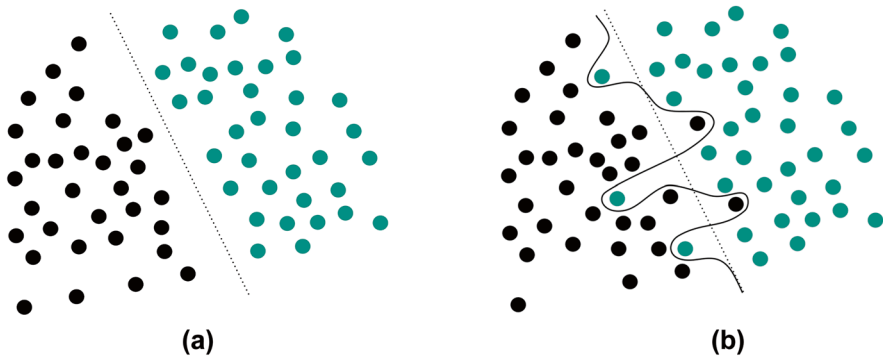


Fig. 4.8 Example of overfitting during the training of the separating surface of two classes. **a)** Idealized data without noise. **b)** Illustration of real data and the effect of overfitting

In section 4.2.3 we discussed the relevance of training and test data. The correct composition of these two sets determines the quality of the training. This is where the so-called cross-validation comes into play, which helps us to make ideal mixtures of data.

We divide the set of all data into k equally large subsets

$$T_1 = (X_1, Y_1), T_2 = (X_2, Y_2), \dots, T_R = (X_k, Y_k), \quad (4.33)$$

We call these subsets **Folds**. Now we train the learning process k times. For this, one Fold T_i is defined, which is used for the test. The remaining $k - 1$ Folds T_m with $m \neq i$ are used for training. This procedure is called **Cross-validation**.

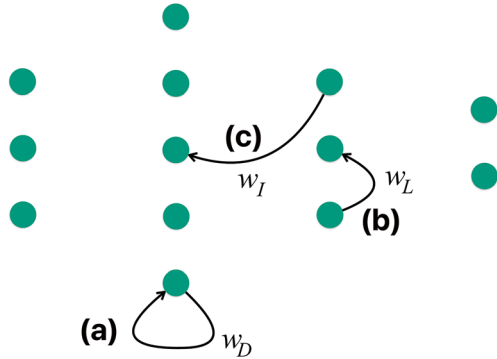
These folds can be quickly and easily generated with a helper function from Scikit-Learn. Listing 4.16 shows, based on already existing data X and labels Y , how to represent the above division process in Python.

Listing 4.16 Generate folds for a cross-validation

```
1 from sklearn.model_selection import KFold
2
3 kf = KFold(n_splits=10)
4
5 for i, j in kf.split(X):
6     Xtrain, Xtest = X[i], X[j]
7     Ytrain, Ytest = Y[i], Y[j]
```

You thus receive sets of X_{train} , X_{test} , Y_{train} and Y_{test} . You now divide your training and test runs onto these sets. The associated iteration of the `fit` call is part of our exercise task 4.7.

Fig. 4.9 Types of feedback in recurrent networks. **a)** Direct feedback, **b)** lateral feedback, and **c)** indirect feedback



4.6 Recurrent Neural Networks

4.6.1 Feedback in Neural Network Structures

Recurrent neural networks deviate from the previous strategy of weight linkage. They allow more complex linkages such as feedbacks of neurons. The following new types of weight connections are distinguished (Fig. 4.9):

- **Direct feedback.** A weight vector is spanned from a neuron onto itself. That is, the output of the neuron is additionally fed back to its input, which generates the feedback. In Fig. 4.9, a recurrent network is shown. Case (a) corresponds to the direct feedback.
- **Lateral (side) feedback.** This leads the output of a neuron to the input of a neuron of the same layer. The flow of information in the network is thus perpendicular to the evaluation direction. The lateral feedback is represented as case (b) in Fig. 4.9.
- **Indirect feedback.** Here the output of a neuron is fed back over one or more layers and connected with the input of a front neuron. This feedback is finally illustrated in diagram 4.9 as case (c).

In the next transformations, we want to illustrate the functioning of recurrent networks. To do this, we simplify our notation. We consider a single neuron with an input x and an output y . Inside, this neuron has the (hidden) internal state h . As with the adaptive filter, we use superscript, bracketed expressions to symbolize the time steps. Due to the recurrence, the same neuron at time t can remember the state at an earlier time step $t - 1$. The internal state $h^{(t)}$ is determined by the input $x^{(t)}$, the weight w_{hx} for this input, the threshold b_h and the described previous value for $h^{(t-1)}$,

$$h^{(t)} = f(w_{hh}h^{(t-1)} + w_{hx}x^{(t)} + b_h). \quad (4.34)$$

The output of the neuron is finally determined by

$$y^{(t)} = f(w_{yh}h^{(t)} + b_y), \quad (4.35)$$

where we use an output weight w_{yh} and another threshold b_y . Along the time axis, the feedback thus has an effect via its own weight w_{hh} .

Since this temporal dimension is of interest for feedback processes, it can be helpful to graphically represent the temporal flow through a neuron. To do this, we move from iteration steps at different times $(t - 1)$, (t) , $(t + 1)$. In Fig. 4.10 we have illustrated this unfolding step.

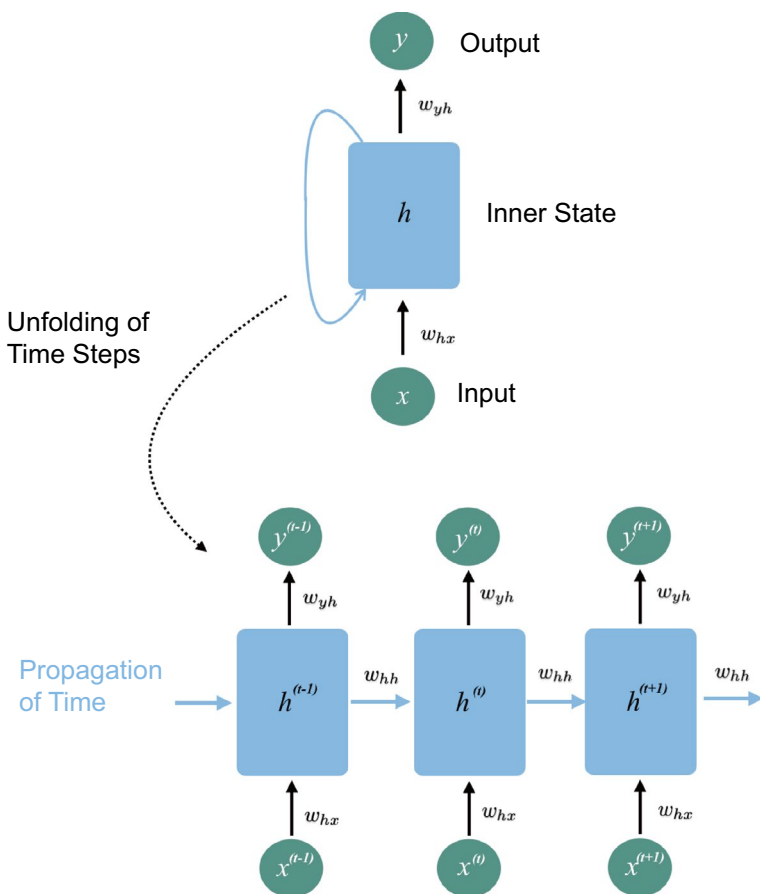


Fig. 4.10 Unfolding of the temporal iteration steps in the recurrent network

4.6.2 *Gates for Controlling the Flow of Information within the Neuron*

Feedback, like we just described, is a fundamental concept of control theory. This data fed back by the network is used in recurrent networks to specifically improve the learning process. However, we need a tool to control the data within a neuron. This tool is the gate. Like a valve, a gate can let data pass or block it. It does not act like a hard switch, but rather like a lock that has a variable opening.

In our case, we define the gate using another sigmoid function. The sigmoid maps its input to the value range between 0 and 1, thus realizing a suitable flow control. We abbreviate the gate with γ and use the index, here A , to refer it to a specific route,

$$\gamma_A(x) = \text{sig}(w_A x). \quad (4.36)$$

The shown gate has its own trainable weight w_A , which is multiplied with the gate input x and finally entered into the sigmoid function. γ_A thus acts as a dynamic weight factor. If this is applied to x_{k-1} ,

$$x_k = \gamma_A(x_{k-1})x_{k-1}, \quad (4.37)$$

a large proportion of x_{k-1} reaches x_k , when γ_A is large, and less when γ_A is correspondingly small. The strength of γ_X thus controls what proportion x_{k-1} has on x_k . Without additional inflows, the single gate does not make much sense yet.

4.6.3 *Neural Network with Long Short-Term Memory (LSTM)*

Next, we want to give a neuron a memory m that lasts over several iterations. This idea goes back to Hochreiter and Schmidhuber [7] and in this case we are talking about a long short-term memory (LSTM). LSTM, along with alternative strategies (e.g., Gated Recurrent Units, GRU), has proven to be extremely successful for application to time series, as also shown by A. Graves in [5]. Speech recognition, automated parsing of texts, and image recognition problems are just a few examples where this technology is used. Therefore, we will specifically deal with the idea behind LSTM and show in the next section how we can use LSTM for our own networks.

The recurrent memory, which we will call m in the course of our considerations, we pass on, just like the hidden state h , from time step to time step. The gates introduced in the previous sections play a special role here. We define three different gates, a forget gate, an update gate, and an output gate. Fig. 4.11 shows a scheme of how the gates are arranged. The temporal change again runs from left to right, with h and m running in this direction. From bottom to top, the mapping from input x to output y is given.

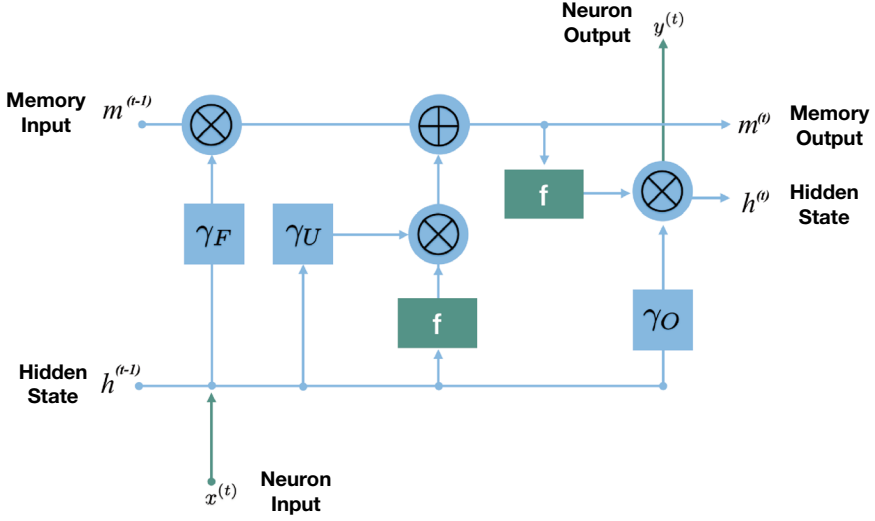


Fig. 4.11 Signal flow through a cell with long short-term memory (LSTM)

The Forget Gate γ_F is determined by

$$\gamma_F = \text{sig}(w_{Fh}h^{(t-1)} + w_{Fx}x^{(t)} + b_i) \quad (4.38)$$

. It is capable of influencing the currently stored value in m . The Update Gate γ_U is given by

$$\gamma_U = \text{sig}(w_{Uh}h^{(t-1)} + w_{Ux}x^{(t)} + b_u) \quad (4.39)$$

and the Output Gate γ_O is calculated by

$$\gamma_O = \text{sig}(w_{Oh}h^{(t-1)} + w_{Ox}x^{(t)} + b_o). \quad (4.40)$$

From the hidden state $h^{(t-1)}$ and the input $x^{(t)}$ a new candidate for the memory can be determined—the potential update for m :

$$c_M = f_k(w_{hh}h^{(t-1)} + w_{hx}x^{(t)} + b_h). \quad (4.41)$$

The new $m^{(t)}$ is determined from the candidate c_M and the old memory value $m^{(t-1)}$,

$$m^{(t)} = \gamma_U c_M + \gamma_F m^{(t-1)}. \quad (4.42)$$

One can imagine this equation as a mixture. Depending on the current pass-through factor of the gates, more of the candidate is added or not. The Forget Gate

allows the memory of the previous memory value $m^{(t-1)}$ to be influenced. If γ_F is low, the memory value loses influence over time.

The network output is finally determined by

$$y^{(t)} = \gamma_O f_k(m^{(t)}), \quad (4.43)$$

where m and h are iteratively passed on over time.

Please note that the above calculations apply to each neuron of a network. In networks with multiple layers, the steps must be performed for each individual neuron. The unfolding should not be confused with the network topology. The illustration in Fig. 4.10 applies to each individual neuron.

4.6.4 Implementation of an LSTM Network

The above equations contain a multitude of new weights, new thresholds, and influences of gates. Nevertheless, these new components of the network can be trained similarly using backpropagation. The complex step of modifying the weights in such a constellation again requires knowledge of the gradient of the cost function with respect to these specific weights. Once this is known, Δw can be determined to change the weights. These complex steps are performed using libraries and do not (fortunately) have to be implemented yourself.

In the following, we always assume networks that are fully recurrent. Libraries like Keras offer us the opportunity to use the capabilities of LSTM neurons for our networks and formulate them recurrently.

For our program example, we prepare training and test data in Listing 4.17. We again use the example file `EXAMPLE02.pickle` from Sect. 3.5.

Listing 4.17 Preparing example data for using LSTM

```

1  x = pickle.load(open('EXAMPLE02.pickle', 'rb'))
2
3  Xtrain = []
4  Ytrain = []
5  Xtest = []
6  Ytest = []
7  windowLength = 25
8  for i in range(0, 5000-windowLength):
9      Xtrain.append([x[i:i+windowLength]])
10     Ytrain.append(x[i+windowLength+1])
11
12  Xtrain = np.array(Xtrain)
13  Xtrain = Xtrain.reshape(Xtrain.shape[0], 25, 1)
14
15  for i in range(5001, 5600-windowLength):
16      Xtest.append([x[i:i+windowLength]])
17      Ytest.append(x[i+windowLength+1])
18
19  Xtest = np.array(Xtest)
20  Xtest = Xtest.reshape(Xtest.shape[0], 25, 1)
```

The aim of Listing 4.17 is to generate a training dataset that always takes a window of length `windowLength` from the variable `x` and finally takes the next following value in `x` as the target value for `Ytrain`. After that, corresponding windows are also built for `Xtest` and `Ytest`.

Listing 4.18 shows the code for the LSTM network. It imports a special layer for Keras that ensures recurrence.

Listing 4.18 Time series prediction using LSTM

```

1  import keras
2  import numpy as np
3  import tensorflow as tf
4  from tensorflow.keras import layers, losses
5  from tensorflow.keras.models import Model
6  from tensorflow.keras.layers import LSTM
7
8  class LSTMPredictor(Model):
9
10     def __init__(self):
11         super(LSTMPredictor, self).__init__()
12         self.inputLayerLength = 1
13         self.myLayers = []
14         self.myLayers.append(layers.LSTM(25, input_shape=(1,
15             25)))
16         self.myLayers.append(layers.Dense(1, activation='relu'
17             ))
18         self.predictor = tf.keras.Sequential(self.myLayers)
19         self.compile(optimizer='adam',
20             loss=losses.MeanSquaredError())
21         self.optimizer.learning_rate = 0.005
22
23     def call(self, x):
24         predictor = self.predictor(x)
25         return predictor

```

With the call in Listing 4.19 we train the network.

Listing 4.19 Running the prediction

```

1  lstmPredictor = LSTMPredictor()
2  history = lstmPredictor.fit(np.array(Xtrain), np.array(Ytrain), epochs
    =120, batch_size=30)

```

Listing 4.20 evaluates the network on the test data and thus predicts the following value for each new window.

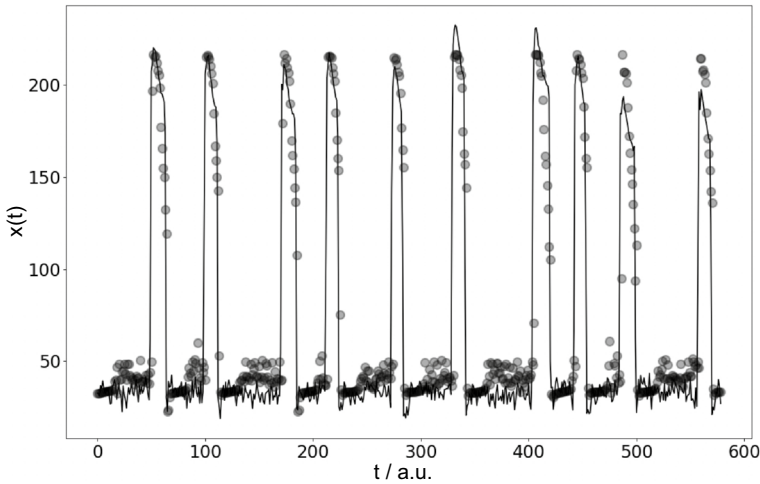


Fig. 4.12 Result of the LSTM prediction on the test data

Listing 4.20 Plotting the result of the time series prediction

```

1 results = lstmPredictor.predict(np.array(Xtest))
2
3 plt.plot(Ytest, 'k-')
4 plt.scatter(range(0, len(results)), results, s=110, marker='o', linewidth
5             =2, color='k', alpha=0.3)
6 plt.tick_params('both', labelsize=22)
7 plt.xlabel('Testfall i', fontsize=24)
8 plt.ylabel('Ytest[i], result[i]', fontsize=24)

```

Fig. 4.12 shows the result of this code. The slightly transparent points represent the prediction of the network. The solid line shows us the true course. The LSTM network thus predicts the future course of the time series for us.

4.7 Decision Trees

4.7.1 Basic Idea of the Decision Tree

We now turn to a conceptually completely different learning method, the decision trees. We all know datasets where we can see at a glance which variables are related. Let's assume we have five columns of data and label them with x_{i1} , x_{i2} , x_{i3} , x_{i4} and x_{i5} . The index i here gives us the i -th data row in a dataset with N different measurements.

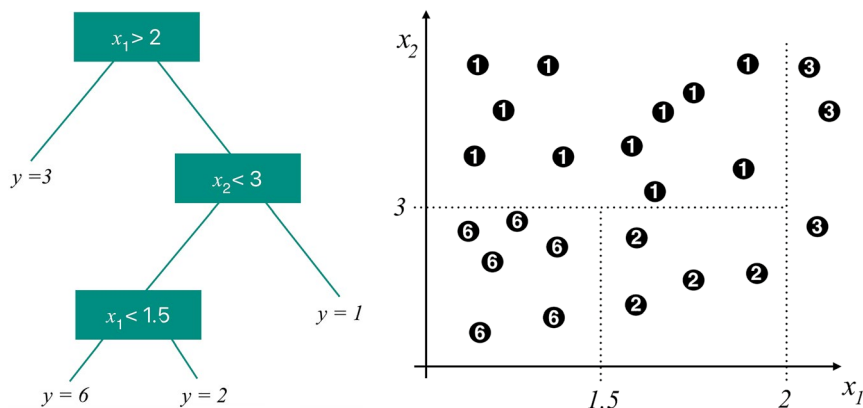


Fig. 4.13 Illustration of what a decision tree might look like (left) and a graphical representation of how the corresponding division in the attribute space X is realized (right). The numbers in the black dots stand for the y value of the target variable

We first define the following terms to be able to express our next steps more clearly in language:

We call a variable y the **target variable** of a procedure if our intention is to be able to predict this variable with the help of the others. All data x_i that we use to predict y , we call **input data** or **attributes**. Here, $x_i \in X$ and $y \in Y$.

Without loss of generality, we assume the variable y to be a scalar value. Thus, reproducing the target variable is the task of this learning method. As before, we also distinguish here between classification and regression. The task of a decision tree is to successively determine the influence of each input variable on the target variable. It statistically determines the probability of the target variable carrying a certain result, depending on the configuration of the input data.

A **decision tree** is a model $f_T : X \rightarrow Y$, which trains rules for the input data $x \in X$ to predict a target variable $y \in Y$.

Fig. 4.13 on the left shows an idealized example tree—here for a quite concrete scenario. The values are only for illustration. On the right side of Fig. 4.13 you can see the segmentation in the data space X , which here also consists of only two variables for illustration. The tree depicted here has correctly divided the various y values by defining rectangular areas in the X space.

Abstractly, an algorithm for such a tree uses a top-down method:

- **Splitting (Split).** Suitable criteria are found on the basis of which a division of the variables can be made. The most important variable, which contains the most information about the prediction variable, is at the beginning of the tree, as root.
- **Pruning (Pruning).** Large tree structures can memorize smallest details instead of correctly training the model. Therefore, the trees are kept as small as possible or pruned.
- **Recursive repetition.** The previous steps are repeated.

4.7.2 Entropy and Information Gain

Example: Branch Optimization

We consider a situation from the branch business. You operate several stores and sell goods. These goods come via a supply route (SupplyChain) and can be stored in a warehouse (Storage). You also have costs for the staff (StaffCost) and generate a certain turnover (Revenue). In Fig. 4.14 various example data are listed. All variables are captured by 0 or 1, which have the following meanings: The warehouse can exist (1) or not (0), the SupplyChain can function well (1) or not (0), the costs for salaries are either high (1) or low (0) and finally the turnover is high (1) or low (0). ◀

The question is first, what is the target variable in this example? Here, of course, the turnover is the obvious choice, because this is the size that we explicitly cannot influence ourselves. It is also the most sensible target size from the problem of optimization. All other variables, on the other hand, are controllable and input variables. The next question is, which of the variables has the strongest influence on the turnover. You can see this, in this selected example, already with the naked eye: the last column and the column for the warehouse (Storage) show a strikingly

	SupplyChain	Storage	StaffCost	Revenue
0	0	1	0	1
1	1	1	1	1
2	1	0	1	0
3	1	1	1	1
4	0	1	1	0
5	0	0	0	0
6	0	0	1	0
7	1	1	1	1
8	0	1	0	0
9	1	1	1	1

Fig. 4.14 Example data for optimizing business branches

high agreement. So one can put forward the thesis that whenever the warehouse is present, the turnover is also high.

This approach is very rudimentary. But it is already very close to the mathematical processes in the decision tree. Now if we count the favorable and possible cases, then there are turnovers with 1 in 6 cases. The table then contains 10 possible cases, so the probability is $6/10 = 1/2$ that a high turnover is present. There are also 7 warehouses. If we limit our focus only to the cases where the warehouse is present, we count all 6 cases with high turnover. Only in one of the 7 warehouse cases was the turnover low. The conditional probability for high turnover, under the condition that a warehouse exists, is therefore

$$P(\text{Revenue} == 1 \mid \text{Storage} == 1) = \frac{6}{7} = 0.85, \quad (4.44)$$

which shows that this variable must have a high influence on the target variable. If a warehouse is present, there is a probability of 85 % that the turnover is high.

Entropy

Let's formalize this idea a bit more. We introduce the size entropy for this purpose.

The **entropy** H of a random variable x , is defined as

$$H(x) = - \sum_{x \in \Theta} P(x_i) \log_2 P(x_i) \quad (4.45)$$

It is a measure of the uncertainty of x . Θ is the event space of x .

Here, the sum runs over all possible states of x .

Example: Coin Toss

When we toss a coin, the possible states of $\Theta = \{\text{Head}, \text{Tails}\}$ and the probabilities $P(\text{Head}) = 0.5$ and $P(\text{Tails}) = 0.5$. The entropy then amounts to $H = -0.5 * \log_2(0.5) - 0.5 * \log_2(0.5) = 1$. ◀

A process that is completely random has maximum uncertainty (maximum disorder). The entropy in this case is $H = 1$. Low values of entropy thus mean a high certainty for the prediction, high values indicate a high randomness. In physics, entropy is also known as a measure of disorder.

For our specific example, we can now calculate the entropy for the turnover, here is $\Theta = \{1, 0\}$. We use the probabilities

$$\begin{aligned} P(\text{Revenue} = 0) &= \frac{4}{10} \\ P(\text{Revenue} = 1) &= \frac{6}{10} \end{aligned} \quad (4.46)$$

and determine the entropy over (4.45) to

$$H_0(\text{Revenue}) = -\frac{6}{10} \log_2 \left(\frac{6}{10} \right) - \frac{4}{10} \log_2 \left(\frac{4}{10} \right) \approx 0.97 \quad (4.47)$$

We call this value H_0 , to indicate that it is a base value of entropy, without considering the other columns of the data. So if we only consider the turnover, the values are almost random. How can we now understand the influence of the other variables on our target size with the help of entropy? Are there variables (columns of the table) that exert an influence on the target column or by which we could predict the target variable?

For this, we limit the data to certain “what if” scenarios. We divide the above table for the variable StaffCost in such a way that two groups are formed, for StaffCost = 0 and StaffCost = 1. In Fig. 4.15 the rearranged tables can be seen. An ideal assignment would be given if our target variable in one half of the table, either top or bottom, only takes the same numerical values. This does not happen here, StaffCost does not directly predict the variable Revenue.

When you calculate the probabilities for the target variable, you determine conditional probabilities—conditioned by the state of the variables based on which you have divided the table. The entropy can also be calculated for this division based on the conditional probabilities and therefore one also speaks of the conditional entropy:

If x and A are two random processes, A can take the states a_i from the event space Θ_A . The **conditional entropy** is then defined as

$$H(x|A) = \sum_{a_i \in \Theta_A} P(a_i) H(x|A = a_i). \quad (4.48)$$

	SupplyChain	Storage	StaffCost	Revenue
0	0	1	0	1
5	0	0	0	0
8	0	1	0	0
	SupplyChain	Storage	StaffCost	Revenue
1	1	1	1	1
2	1	0	1	0
3	1	1	1	1
4	0	0	1	0
6	0	1	1	1
7	1	1	1	1
9	1	1	1	1

Fig. 4.15 Division of data rows for low and high StaffCost values

In a clear way, the conditional entropy thus goes through all states of the variable A . It uses the entropy in each subgroup of the data and weights each state with its probability of occurrence $p(a_i)$. We test this again in our example above. First, we consider the category “StaffCost”. The conditional probabilities for this can be read off in Fig. 4.15,

$$\begin{aligned} P(\text{Revenue} = 1 | \text{StaffCost} = 0) &= \frac{1}{3}, \\ P(\text{Revenue} = 0 | \text{StaffCost} = 0) &= \frac{2}{3}, \end{aligned} \quad (4.49)$$

which ultimately leads to an entropy

$$H(\text{Revenue} | \text{StaffCost} = 0) = -\frac{1}{3} \log_2 \left(\frac{1}{3} \right) - \frac{2}{3} \log_2 \left(\frac{2}{3} \right) \approx 0.91. \quad (4.50)$$

. Similarly, the entropy for the second half of the table is found,

$$H(\text{Revenue} | \text{StaffCost} = 1) = -\frac{5}{7} \log_2 \left(\frac{5}{7} \right) - \frac{2}{7} \log_2 \left(\frac{2}{7} \right) \approx 0.86. \quad (4.51)$$

Both are the entropies in the summands of (4.48). The weights $P(a_i)$ in 4.48 are the probabilities that StaffCost takes either the value 1 or 0, and we can count these just as easily. The total conditional entropy $H(\text{Revenue} | \text{StaffCost})$ is thus (Fig. 4.15)

$$H(\text{Revenue} | \text{StaffCost}) = \frac{3}{10} * 0.91 + \frac{7}{10} * 0.86 = 0.875. \quad (4.52)$$

The StaffCost column thus has a lower entropy for determining the target variable Revenue than if we had only looked at the target variable. Now let’s look at the variable Storage, which we have already identified as extremely influential, resulting in a conditional entropy of

$$H(\text{Revenue} | \text{Storage}) = \frac{3}{10} * 0.0 + \frac{7}{10} * 0.59 = 0.413, \quad (4.53)$$

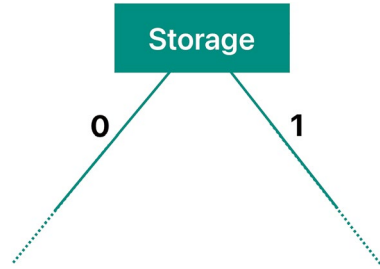
even lower than for StaffCost. The information gain through the column is therefore higher than through StaffCost. We can therefore determine the influence of a variable on the prediction of our target size by the conditional entropy.

Information Gain

If we reduce the entropy, we gain information. Exactly this relationship can be described by the following new size (Fig. 4.16):

We define the **information gain** (Information Gain) IG by the variable A , as the distance between the original entropy $H_0(x)$ for the prediction of a target variable x , and the conditional entropy $H(x|A)$,

Fig. 4.16 First node of a decision tree



$$IG(x, A) = H_0(x) - H(x|A). \quad (4.54)$$

The **Information Gain** (IG) gives us the distance from the base entropy of a target variable to the conditional entropy by restricting to an input variable. The greater this distance, the more information the corresponding column has contributed to our criterion. For the above example, the following values result

$$\begin{aligned} IG(\text{Revenue}|\text{StaffCost}) &= 0.97 - 0.875 \approx 0.095, \\ IG(\text{Revenue}|\text{Storage}) &= 0.97 - 0.413 \approx 0.56, \\ IG(\text{Revenue}|\text{SupplyChain}) &\approx 0.12. \end{aligned} \quad (4.55)$$

This result confirms the impression that Storage has the most influence on the result. The first node is therefore Storage, see Fig. 4.16. Here the information gain is greatest. Accordingly, the tree is built up iteratively, as Fig. 4.17 shows.

Construction of a simple decision tree by division

To create a decision tree, we now follow an algorithm of the form:

- **(1) Calculate IG.** In a dataset, the IG for all input variables with respect to a target variable is calculated.
- **(2) Leaf.** The input variable with the highest IG is set as a node of a decision tree. The influence of this size is thus processed.
- **(3) Branching.** Through the tree root, two new datasets are created that correspond to the division based on the input variables.
- **(4) Iteration.** For each new data table, we iteratively return to step (1) and continue the process until all variables are processed.

The (simplified) algorithm shown here goes back to R. Quinlan [11] and is called Iterative Dichotomizer (ID). In fact, we only consider the process of division. The algorithms C4.5 and C5.0 (both decision trees) also use a division over the IG. It is perhaps the most illustrative approach to understanding the construction of decision trees. There is no restriction on the number of splits, which unfortunately can lead to broad tree structures. Our example was chosen in such a way that this effect cannot occur here.

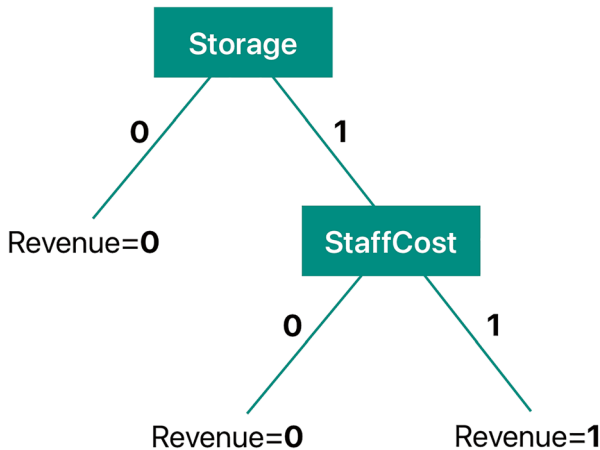


Fig. 4.17 First node of a decision tree

4.7.3 Classification and Regression Trees

The branching into many branches from a node is a disadvantage of the above method, as it promotes wide tree structures. L. Breiman et al. showed in [2] an approach based on binary divisions: the Classification-and-Regression-Tree algorithm (CART). Each node can only develop two branches here.

To specifically train a decision tree, a cost function J is again needed. It describes the deviation of the tree output y to the desired label d (be it a classification or a regression),

$$J = \sum_{i=0}^{N-1} [f_T(\mathbf{x}_i) - d_i]^2 \quad (4.56)$$

starting from a dataset with N data rows $\mathbf{x}_i$ and N labels y_i . This cost function is already well known to us, it is again the mean square distance.

To counteract any growth of the tree, a penalty term can be added to the cost function. This term is supposed to increase the costs if the tree becomes too large. If K is the number of tree branches, then we choose this factor as $R = \alpha K$.

In summary, we can write the training of a decision tree as:

A decision tree is trained via an algorithm that divides the space X into K disjoint subsets $\mathcal{X}_k$,

$$X = \mathcal{X}_0 \cup \mathcal{X}_1 \cup \dots \cup \mathcal{X}_K, \quad (4.57)$$

so that over

$$\begin{aligned} &\text{minimize} && J = \sum_{i=0}^{N-1} (f_T(\mathbf{x}_i) - d_i)^2 + \alpha K \\ &\text{u. d. B.} && \mathbf{x}_{ik} \in \mathcal{X}_k \wedge \mathcal{X}_k \cap \mathcal{X}_m = \emptyset \text{ für } k \neq m, \\ &&& \text{the cost function } J \text{ is minimized.} \end{aligned} \quad (4.58)$$

Each vector $\mathbf{x}_i$ thus consists of k entries. Through the arrangement of the $\mathcal{X}_k$ of the tree, this minimization ultimately results in a model with $y = f_T(\mathbf{x}) \approx d$.

4.7.4 Application Example with Scikit-Learn

We now want to deal with the application of such a decision tree. In Python, there are several libraries that support us in training such tree structures.

Listing 4.21 Decision tree classifier with Scikit-Learn

```

1  %matplotlib tk
2  import matplotlib
3  import matplotlib.pyplot as plt
4  from sklearn.tree import DecisionTreeClassifier
5  from sklearn import tree
6  import numpy as np
7  from scipy import stats
8
9  import pandas as pd
10
11 company = pd.DataFrame()
12 company['SupplyChain'] = [0,1,1,1,0,0,0,1,0,1]
13 company['Storage'] = [1,1,0,1,0,0,1,1,1,1]
14 company['StaffCost'] = [0,1,1,1,1,0,1,1,0,1]
15 company['Revenue'] = [1,1,0,1,0,0,1,1,0,1]
16 cpm = pd.DataFrame(company)
17
18 X = []
19 for i in range(0, len(company['Storage'])):
20     x = [company['StaffCost'][i], company['SupplyChain'][i],
21         company['Storage'][i]]
22     X.append(x)
23
24 Y = company['Revenue']
25
26 clf = DecisionTreeClassifier(criterion='entropy')
27 clf.fit(X, Y)
28 fig = plt.figure(figsize=(12,4), dpi=100)
29 tree.plot_tree(clf)

```

Listing 4.21 shows the application of the decision tree from the Scikit-Learn library. The tree can be graphically output and interpreted using the function `tree.plot_tree`.

The binary entropy is needed to calculate the balance for the division of the set `ytrain`. We divide this set using a boolean array `b` into a “left” part and a “right” part. The left branch thus takes the `y` values where the value `b` was `True`, the right branch takes the remaining set. In Listing 4.24, this part of the calculation is shown and a total entropy value H for a specific vector `b` is determined.

Listing 4.24 Distribution using binary entropy

```

1  def H(self, b:bool, ytrain:np.array):
2      left = ytrain[z]
3      right = ytrain[~z]
4      nLeft, hLeft = 0,0
5      nRight, hRight = 0,0
6      for yd in set(left):
7          nLeft = sum(left==yd)
8          hLeft += float(nLeft)/self.N * self.binaryEntropy(
9              sum(left==yd), sum(left!=yd))
10     H = float(nLeft)/self.N * hLeft
11     for yd in set(right):
12         nRight = sum(right==yd)
13         hRight += float(nRight)/self.N * self.
14             binaryEntropy(sum(right==yd), sum(right!=yd))
15     H += float(nRight)/self.N * hRight
16     return H

```

With the help of these two subfunctions, we can perform the split of the training and target variables in Listing 4.25. This split works in such a way that we iterate through each `x` value and use `b` to minimize the entropy. The cutoff is the `x` value at which we divide the branches as optimally as possible.

Listing 4.25 Performing the branch split with binary entropy

```

1  def split(self, xtrain:np.array, ytrain:np.array):
2      column = None
3      cutoff = None
4      entropyMinimum = 100
5      localEntropyMinimum = 1
6      for i, xc in enumerate(xtrain.T):
7          for value in set(xc):
8              b:bool = xc < value
9              theEntropy = self.H(b, ytrain)
10             if theEntropy <= localEntropyMinimum:
11                 localEntropyMinimum = theEntropy
12                 theCutoff = np.round(float(value),4)
13
14             if theEntropy == 0:
15                 return i, theCutoff, theEntropy
16
17             elif theEntropy <= entropyMinimum:
18                 entropyMinimum = theEntropy
19                 column = i
20                 cutoff = np.round(float(theCutoff),4)
21
22     return column, cutoff, entropyMinimum

```

While `split` calculates a division for a specific dataset of the training set, the function `fit` in Listing 4.26 actually goes through all the data of the training set. This is where the actual tree is created. In line 12, the above `split` is called. Afterwards, the data is divided according to the results determined here for the `cutoff`. Lines 15–20 show the creation of each node of the tree. Here, we use the median of the `y` values for the calculation of the leaf value.

The most important step is finally the recursion for the left and right branch, where we again call `fit`. The process is thus continued in a nested manner and only stopped when the specified maximum tree depth has been reached.

Listing 4.26 Fitting the data into the tree

```

1  def fit(self, xtrain:np.array, ytrain:np.array, node={},
2      depth:int=0):
3      self.N = len(ytrain)
4      if node is None:
5          return None
6      elif np.shape(ytrain)[0] == 0:
7          return None
8      elif all(x == ytrain[0] for x in ytrain):
9          return {'Leaf': np.round( float(ytrain[0]), 4) }
10     elif depth >= self.treeDepthMaximum:
11         return None
12     else:
13         col, cutoff, entropy = self.split(xtrain, ytrain)
14         ytrain1 = ytrain[xtrain[:, col] < cutoff]
15         ytrain2 = ytrain[xtrain[:, col] >= cutoff]
16         node = {
17             'Index':col,
18             'Cutoff':cutoff,
19             'Leaf': np.round(np.median(ytrain))
20         }
21         node['Left'] = self.fit(xtrain[xtrain[:, col] <
22             cutoff], ytrain1, {}, depth + 1)
23         node['Right'] = self.fit(xtrain[xtrain[:, col] >=
24             cutoff], ytrain2, {}, depth + 1)
25         self.treeDepth += 1
26         self.tree = node
27         return node

```

The tree described here can already be trained with the given functions. However, it cannot yet be tested, as we have not yet created access to meaningfully read the tree again. In Listing 4.27, we therefore supplement the function `predict`. It takes test data `xtest` and goes through the tree according to its branching. To do this, we iterate over each data vector that is present in the set `xtest`. Afterwards, we use the `x` value to iterate to the corresponding position of the tree and find the last leaf that contains the target value.

Listing 4.27 Using the tree to predict results

```

1  def predict(self, xtest:np.array):
2      results = []
3      for eachTest in xtest:
4          classResult = 0
5          tree = self.tree
6          check = True
7          lastLeaf = 0
8          while check:
9              if eachTest[tree['Index']] < tree['Cutoff']:
10                 tree = tree['Left']
11             else:
12                 tree = tree['Right']
13             if tree!=None:
14                 if 'Leaf' in tree:
15                     lastLeaf = tree['Leaf']
16                 if 'Index' in tree:
17                     check = True
18                 else:
19                     check = False
20             else:
21                 check = False
22         else:
23             results.append(lastLeaf)
24     return results

```

If you put the tree together into an object `simpleTree()`, we can train this tree with the code in Listing 4.28. Here we again use our established motor current example, which includes a classification.

Listing 4.28 Calling the simple tree and applying it to an example

```

1  import pickle
2  data = pickle.load(open('EX03Engine.pickle','rb'))
3  X = data['X']
4  Y = data['Label']
5  clf = simpleTree(maxDepth=10)
6  clf.fit(np.array(X[0:80]),np.array(Y[0:80]))

```

To perform the test, we use a range of data that we have not used for training and revert to our simple verification of the results. Listing 4.29 shows how the test is called.

Listing 4.29 Test of the simple tree

```

1  result = clf.predict(np.array(X[100:120]))
2  for i in range(0,19):
3      print('{} | {}'.format(np.round(result[i],2), Y[100+i]))

```

The expected test result of our simple tree leads to a good classification, as the output in 4.30 illustrates.

Listing 4.30 Result of simple tree

```

1  1 | 1
2  2 | 2
3  3 | 3
4  2 | 2
5  1 | 1
6  2 | 2
7  2 | 2
8  2 | 2
9  2 | 2

```

The simple tree shown in the previous section can serve as a starting point for your own extensions. It illustrates the basic concept and explains the division of the branches. The core element of the division was the calculation of entropy. Alternatively, you could certainly also maximize the Information Gain (IG), which leads to equivalent results.

4.7.6 The Gini Coefficient for Decision Trees

The Gini coefficient, named after Corrado Gini [4], is a measure of concentration for sets. It is, alternatively to entropy, a popular tool for determining the quality of mixtures. The Gini index, as it is also called, indicates the deviation of data points from a uniform distribution. It therefore offers another way to determine the quality of a division.

The **Gini coefficient** is defined as

$$G(\mathbf{a}) = \sum_{a_i \in \Theta_A} p(a_i)(1 - p(a_i)). \quad (4.59)$$

and indicates how much the values x_i deviate from $\mathbf{x}$ a uniform distribution.

In the code in Listing 4.31, we calculate the Gini coefficient for a binary division. This will later allow us to use the code in our above decision tree.

Listing 4.31 Gini coefficient

```

1  def gini(x):
2      a = np.sum(x==True)
3      b = np.sum(x==False)
4      pa = float(a)/(a+b)
5      pb = float(b)/(a+b)
6      g = pa*(1-pa) + pb*(1-pb)
7      return g

```

However, we first want to experiment in Listing 4.32 to see how the Gini coefficient behaves.

Listing 4.32 Understanding the Gini coefficient

```

1  x = []
2  x.append(np.array([True, True, True, True, True, True]))
3  x.append(np.array([False, True, True, True, True, True]))
4  x.append(np.array([False, False, True, True, True, True]))
5  x.append(np.array([False, False, False, True, True, True]))
6  x.append(np.array([False, False, False, False, True, True]))
7  x.append(np.array([False, False, False, False, False, True]))
8  x.append(np.array([False, False, False, False, False, False]))
9
10 for eachX in x:
11     print(gini(eachX))

```

The output of these examples yields the values in Listing 4.33.

Listing 4.33 Result of Listing 4.32

```

1  0.0
2  0.2777777777777778
3  0.4444444444444445
4  0.5
5  0.4444444444444445
6  0.2777777777777778
7  0.0

```

The Gini coefficient in this implementation takes values between 0 and 0.5. To use it instead of entropy in the decision tree, we integrate the following listing:

Listing 4.34 Binary Gini coefficient for use in our tree

```

1  def binaryGini(self, A:int, B:int):
2      pa = float(A)/(A+B)
3      pb = float(B)/(A+B)
4      g = pa*(1-pa) + pb*(1-pb)
5      return g

```

This function has the same call as the `binaryEntropy` function, so the tree can easily be switched to the alternative measure: You now only need to replace the call of the `xtest` with `binaryGini`.

Summary

In this chapter, we first dealt with the topic of learning in general and finally defined computer-aided learning as the result of an optimization process. We get to know two significant strategies in this book: supervised and unsupervised learning. In the course of this chapter, we focused on supervised learning, which learns a function f with $y = f(x)$, where data for x and y are available. The y are the labels or the target values we train on, and x are the input data.

The evolutionary algorithm learns by pure trial and error. It knows, at least in the variant shown here, no better strategy than to vary and test. This strategy is

very similar to that of the game. As simple as this approach appears, this algorithm has already been successfully used in technical applications. Therefore, it was important for us to present this procedure here.

The LMS algorithm is a fundamental concept in control theory. It changes its filter parameters and thus learns to achieve a target size. Due to its simple structure, it is well suited as a first example of learning methods. Its basic training is very similar to that of the neural network.

Then we introduced neural networks. They play a prominent role in today's machine learning. Neural networks train weight and bias values in such a way that the network performs the correct mapping from input to output data.

Finally, we discussed decision trees. Here we first learned concepts for building the trees: entropy and Gini coefficient. Then we discussed both the Scikit-Learn decision tree and programmed our own Hello-World decision tree.

After this chapter, you should be familiar with the basic terms for learning methods. The aim here is not to present the full range of all possible methods, but to discuss the most important ideas and concepts using clear programming examples.

Tasks

4.1 Insert a counter into the code of the evolutionary algorithm that records the number of iterations in the `while` loop. Then add another counter to record the number of successful change processes. What is the ratio of the two numbers when you calculate multiple runs?

4.2 Test various activation functions in the implementation of the regression network: a) Sigmoid, b) Tanh, and c) ReLU. What happens to the learning curve? What influence does the choice of activation function have on the quality, in relation to this specific example?

4.3 Apply the Scikit-Learn implementation of the decision tree to our problem with the motor currents. What difference do you notice, by switching from Gini coefficients to an entropy-based distinction?

4.4 How could you integrate the quality function J from (4.58) into our simple decision tree `simpleTree`? At which points would you need to change the code?

4.5 Revisit the classification network with Keras from 4.5.6 and plot the learning curves for the same network, but with a) ADAM optimizer, b) RMSProb optimizer, and c) ADAGRAD optimizer.

4.6 Plot the learning curve for the regression network in Keras from 4.5.8.

4.7 Modify the training and test data for the LSTM network in Sect. 4.6.4 to model the motor current example data. Note: First, string all data vectors together to form a single, long time series. Then use the split demonstrated in Sect. 4.6.4 before you train the network.

4.8 Train the regression network from 4.5.8 using cross-validation! To do this, write the call to training (`fit()`) in a loop and successively evaluate the test results. Vary the test results between the folds.

4.9 Integrate the Gini coefficient into the `simpleTree`. To do this, replace the call to `binaryEntropy` in Listing 4.24 with the call to the function `binary-Gini` from 4.34.

References

1. C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford University Press, 1996.
2. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Taylor and Francis, 1984.
3. J. Duchi, E. Hazan, and Y. Singer, „Adaptive subgradient methods for online learning and stochastic optimization,“ *Journal of Machine Learning Research*, vol. 12, no. Jul, pp. 2121–2159, 2011.
4. C. Gini, „Measurement of inequality of incomes,“ *The Economic Journal*, vol. 31, no. 121, pp. 124–, 1921.
5. A. Graves, „Generating sequences with recurrent neural networks,“ 2014.
6. S. Haykin, *Adaptive Filter Theory*. Prentice-Hall, 2002.
7. S. Hochreiter and J. Schmidhuber, „Long short-term memory,“ *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
8. D. Kingma and J. L. Ba, „Adam: A method for stochastic optimization,“ in *International Conference on Learning Representations*. International Conference on Learning Representations, 2015.
9. S. Koike, „A new efficient method of convergence calculation for adaptive filters using the sign algorithm with digital data inputs,“ *Acoustics, Speech, and Signal Processing, IEEE International Conference on*, vol. 3, p. 2333, 1997.
10. Tinterwordspacing J. Quinlan, „Simplifying decision trees,“ *International Journal of Man-Machine Studies*, vol. 27, no. 3, pp. 221–234, 1987. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020737387800536>
11. R. Quinlan, *Learning efficient classification procedures*. Springer, 1983.
12. M. Riedmiller and H. Braun, „A direct adaptive method for faster backpropagation learning: the rprop algorithm,“ in *IEEE International Conference on Neural Networks*, 1993, pp. 586–591 vol.1.
13. B. D. Ripley, *Pattern Recognition and Neural Networks*, 1st ed. Cambridge University Press, 2008.
14. S. Ruder, „An overview of gradient descent optimization algorithms,“ *arXiv preprint arXiv:1609.04747*, 2016.
15. B. Widrow and M. Kamenetsky, *Least-Mean-Square Adaptive Filters*, S. Haykin and B. Widrow, Eds. Wiley, 2003.

Chapter 5

Unsupervised Learning



Keywords Unsupervised learning • Principal component analysis • K-Means clustering • Autoencoder • Anomaly detection

Unsupervised learning seeks to uncover structures in the data. Unlike supervised learning, there are no labels or target sizes in unsupervised learning. Only the data itself is the starting point of an unsupervised learning process and therefore these methods are excellently suited for preprocessing. Even more: through their special analysis of the data, some of the methods find simpler, low-dimensional representations of the input data. They thus reduce the dimension and enable a better view of the relevant information. Our focus in this chapter will be on the classic Principal Component Analysis (PCA), the K-Means clustering method, the t-Distributed-Stochastic-Neighbour-Embedding algorithm (t-SNE) and the concept of autoencoders.

Unsupervised learning analyzes the structure in data. In doing so, it can extract important information for supervised learning: the labels or target sizes. Even more important is its ability to learn efficient transformations, such as the PCA or other dimension-reduced representations. This leads to a new form of data preprocessing. Chapter 5 links back to Chap. 3 and indirectly provides optimal input data for supervised learning methods.

Fig. 5.1 provides an overview of the impact of Chap. 5 on the other sections of the book. It also influences explainability, as understanding the structure of input data improves the explainability of models.

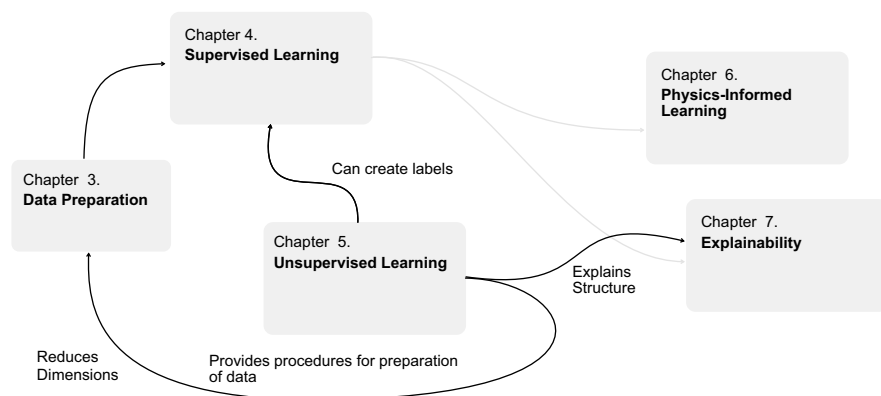


Fig. 5.1 Overview of the connection of Chap. 4 with the following chapters

5.1 Unsupervised Learning Paradigms

Creating models for variables is an obvious strength of supervised learning. It is based on the provision of input data and desired output behavior, input and labels, so that a model learns the relationship between the two. In many cases, however, we do not have labels for the training. Imagine a dataset where you have not yet selected any target quantity. Or think of a situation where you have collected process data but have no information about whether the products produced are faulty or not.

In all these cases, there is no specification of a label and no target variable for a regression. However, there are learning methods that can say something about the data even in this situation. They systematically establish relationships between the data points and try to identify an underlying structure—if there is one. In some cases, these approaches are able to reduce data in their dimension. The latter is also an idea that runs through the preprocessing steps from Chap. 3. Please note, all unsupervised learning methods can also be used for preprocessing for supervised learning. The following methods belong to the group of unsupervised learning and will be in the focus of this chapter:

- **Principal Component Analysis.** Perhaps the best-known method for preprocessing data is the Principal Component Analysis. It uses a suitable transformation, the Karhunen-Loevé transformation, to order the importance of the respective eigenvectors in the eigen space of the covariance matrix and to specifically delete them. This creates a low-dimensional subspace that adequately represents our data.
- **K-Means Clustering Method.** A possible structure formation in data are clusters, i.e., data points that lie close together in larger groups. Determining such clusters is important as it helps us to find out to what extent different regimes